PriDFL: Computation-Optimized Secure Aggregation With Byzantine-Resilient in Decentralized Federated Learning

Shuai Wang, Youliang Tian, *Senior Member, IEEE*, Jinbo Xiong, *Senior Member, IEEE*, Renwan Bi, *Member, IEEE*, Jianfeng Ma, *Member, IEEE*, and Yan Zhang, *Fellow, IEEE*

***Abstract*—Privacy-preserving federated learning (PPFL) is a strong-privacy distributed learning paradigm, which typically employs secure aggregation (SA) protocols to protect the aggregation results of federated learning. However, the high computational cost of existing SA protocols is difficult to generalize to decentralized federated learning (DFL) with the large number of clients and fails to defend against model poisoning attacks launched by Byzantine adversaries. In this paper, we propose an efficient computational SA protocol compatible with DFL, referred to as PriDFL, and address the issue of Byzantine-robust aggregation. Specifically, we design an advanced secret-sharing protocol based on number-theoretic transformations to reduce the computational complexity from $O(n^2)$ to $\mathcal{O}(n \log n)$ during data sharing. We employ a single-mask approach to provide lightweight gradient privacy protection for DFL. To mitigate the impact of poisoned gradients on model convergence, we develop a Byzantine resilience criterion grounded in model cross-updating. The proposed criterion efficiently detects poisoned gradients and non-independent identically distributed (non-IID) data with local computation. Security analysis shows that PriDFL satisfies the security requirements in an honest but curious setting. We evaluate PriDFL on typical datasets (e.g., MNIST and CIFAR-10) and the results show that PriDFL is computation-communication efficient and Byzantine resilient. In particular, PriDFL optimizes the computational efficiency by 6-10$\times$ compared to the well-utilized SA protocols while supporting Byzantine robustness.**

***Index Terms*—Decentralized federated learning, privacy-preserving federated learning, robust federated learning, secure aggregation.**

I. Introduction

FEDERATED learning (FL) represents a cutting-edge approach to distributed learning, allowing for the collaborative training of models across numerous devices while preserving local data privacy. However, one notable limitation is that gradients can inadvertently reveal the local data distribution in FL. To mitigate this issue, privacy-preserving federated learning (PPFL) has gained traction in industry and academia. PPFL leverages cryptography, secure multi-party computation, and differential privacy to protect the model gradients, thereby preventing significant information disclosure caused by gradient inversion in FL systems.

Existing PPFL research predominantly focuses on architectures involving centralized server collaboration. However, centralized structures are plagued by communication bottlenecks [1], single points of failure [2], and significant privacy trust risks [3]. Malicious servers controlled by Byzantine adversaries may tamper with or forge aggregated intermediate values, thereby compromising the convergence of the global model [4]. Furthermore, client dropouts due to unreliable communication links are common when large numbers of clients collaborate to train FL models. Additionally, resource-constrained mobile devices often lack the computational power required for complex tasks. To address these issues, we integrate the decentralized structure into PPFL to develop a novel paradigm of privacy-preserving decentralized federated learning (PPDFL). PPDFL effectively addresses the aforementioned challenges. The secure aggregation (SA) protocol is the security core of PPFL, and the primary challenge in implementing PPDFL is designing a SA protocol that conforms to decentralized architectures.

Since the existence of low-computing power mobile devices and unstable communication environments in real-world scenarios, the SA protocol was first proposed by Bonawitz et al [5]. SA employs a double-masked blinding protocol and Shamir's secret sharing [6] to protect clients' local data privacy and enhance robustness against client dropouts. Numerous studies

Received 17 November 2024; revised 5 November 2025; accepted 17 December 2025. Date of publication 22 December 2025; date of current version 12 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62272123 and Grant 62272102, in part by the Project of High-level Innovative Talents of Guizhou Province under Grant 20206008, in part by the Science and Technology Program of Guizhou Province under Grant 20205017 and Grant 2022065, in part by the Natural Science Foundation Key Program of Fujian Province under Grant 2023J02014, and in part by the Minjiang University Scientific Research Startup Fund for Introduced Talents under Grant MJY25019. *(Corresponding authors: Youliang Tian; Jinbo Xiong.)*

Shuai Wang is with the State Key Laboratory of Public Big Data, College of Computer Science and Technology, Guizhou University, Guiyang 550025, China (e-mail: wshuai957@gmail.com).

Youliang Tian is with the State Key Laboratory of Public Big Data, College of Big Data and Information Engineering, Guizhou University, Guiyang 550025, China (e-mail: youliangtian@163.com).

Jinbo Xiong is with the Fujian Provincial Key Laboratory of Network Security and Cryptology, College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China (e-mail: jbxiong@fjnu.edu.cn).

Renwan Bi is with the School of Cyber Science and Engineering, Minjiang University, Fuzhou 350108, China (e-mail: brw2806@163.com).

Jianfeng Ma is with the State Key Laboratory of Integrated Service Networks (ISN), School of Cyber Engineering, Xidian University, Xi'an 710126, China (e-mail: jfma@mail.xidian.edu.cn).

Yan Zhang is with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: yanzhang@uestc.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2025.3646726, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2025.3646726

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

have focused on reducing computational cost, decreasing communication overhead, and enhancing the verifiability of SA. Specifically, previous work [7] has improved the computational efficiency of SA by designing a new secret-sharing protocol based on the Fast Fourier Transform. Bell et al [8]. reduced the communication overhead of SA by employing incomplete graphs to alter the frequency of client interactions in SA. Additionally, many studies have noted that SA is vulnerable to malicious servers forging aggregation results and have provided verifiability for SA based on cryptographic theory [9], [10], [11], [12]. However, the computational cost and communication overhead of SA increase significantly with the number of clients, limiting the scalability of SA. Furthermore, the SA protocol in PPFL requires coordination by servers to eliminate masks, which is contrary to the principles of DFL.

In addition, the primitive SA protocol fails to consider the fact that the local black-box training in PPFL is invisible and extremely vulnerable to poisoning attacks to corrupt the performance of the federated global model. Machine learning (ML) communities widely focus on strategies to defend against poisoning attacks in the training phase, which are mainly categorized into three wapoisoning filtering [13], model normalization [14], [15], and robust training [16]. Meanwhile, an alternative approach that calculates model similarity based on distance functions (e.g., cosine distance [17], Euclidean distance [18], and Manhattan distance [19]) has emerged, which mainly eliminates outlier models by comparing the distances computed from the direction of model updates. Unfortunately, the above scheme relies on a model of client-to-client interaction to select optimal gradients. This model is incompatible with PPDFL, as it discloses client gradient privacy and imposes an immeasurable communication overhead due to cross-client interaction computations. Therefore, it is crucial to design a low-overhead and secure Byzantine-robust strategy for PPDFL.

To design a secure and efficient SA protocol suitable for PPDFL, we distill the following four technical dilemmas: ① How to design computation-communication efficient SA protocols on resource-constrained mobile devices? ② How to design Byzantine-robust SA methods for PPDFL? ③ How to ensure that SA protocol supports client dropouts without privacy disclosure? ④ How to guarantee the correctness of SA protocols against non-independently identically distributed (non-IID) data from heterogeneous devices? To tackle the aforementioned challenges, we introduce PriDFL, a practical and efficient SA protocol. PriDFL offers a faster secret-sharing mechanism, facilitating secure and efficient gradient data sharing among clients, while mitigating the privacy risks associated with client dropouts. Notably, we integrate a single-mask mechanism within random communication, prioritizing the security of local data within the PPDFL framework. To fortify the defense against model poisoning without requiring client interaction, we devise a Byzantine-resilience strategy centered on sign distribution and KL-divergence analysis of cross-model updates. The main contributions of this paper are summarized as follows:

- We propose a practical and efficient SA protocol named PriDFL. PriDFL employs the proposed faster secret sharing protocol based on number-theoretic transformation (NTT) for data sharing, which reduces the traditional SA protocol complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ while ensuring the threshold privacy.
- We provide a lightweight single-mask approach in a randomized communication architecture, which guarantees that the client learns only the aggregation result and no other additional information by masking with one-time pads.
- We design a Byzantine-resilience aggregation criterion that effectively prevents model poisoning initiated by Byzantine adversaries. In particular, we mitigate the impact of model poisoning using KL-divergence to evaluate the sign distribution of cross-model updates, and clients do not interact with each other without risking privacy disclosure.
- We evaluate the performance of PriDFL in multiple dimensions and typical datasets. Experiments show that PriDFL is computation-communication efficient and Byzantine robust. Specifically, PriDFL improves the computational efficiency over SecAgg [5] by 6-10$\times$ when sharing data.

The rest of this paper is organized as follows: related works and preliminaries are presented in Sections II and III. In Section IV, we describe the notation, system architecture, security model and design goals, respectively. In Section V, we demonstrate the core components of PriDFL: Faster-sharing, single masking in random communication, and Byzantine resilient aggregation. In Section VI, we conduct a theoretical analysis of PriDFL from three perspectives: security, computational/communication efficiency, and correctness. Section VII evaluates the efficiency of PriDFL in detail in several dimensions. Next, we discuss the limitations and solutions for training frameworks and trusted entities in practical deployment within Section VIII. Finally, Section IX summarizes the entire work.

II. Related Work

A. Secure Aggregation for Privacy-Preserving Federated Learning

Existing researches implement SA in PPFL primarily via cryptographic methods [20], [21], trusted-execution environment [22], differential privacy [23], [24]. In this paper, we focus on cryptography-based schemes for SA protocols.

Cryptography-based SA protocols apply cryptographic schemes to encrypt the client's local gradient, ensuring that the client only learns the aggregated global model without revealing their local gradient. There are three main types of cryptography-based SA protocols: masking, additive homomorphic encryption (AHE), and functional encryption. Bonawitz et al. [5] proposed SecAgg, a mask-based SA protocol utilizing a double-masking approach. SecAgg supports client dropout without disclosing privacy, enhancing the practicality of SA [25], [26]. In Section V-A. A., we optimize this type of mask-based SA protocol to support user dropout as well. Additionally, some prior work [27], [28], [29] leverages cryptographic tools based on SecAgg to prevent malicious servers from tampering with aggregation results, thereby increasing the verifiability of SA. Due to the homomorphic properties of AHE, some solutions [30], [31]

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

use multi-key AHE to achieve ciphertext summation with local gradients. Functional encryption-based SA protocols primarily employ the multi-input functional encryption scheme [32] to aggregate the global model while protecting the privacy of the gradients [33], [34]. However, these solutions require servers to coordinate the elimination of secret mask values, which is obviously not applicable to DFL. Hence, it is challenging to customize a lightweight privacy-preserving means for DFL to achieve gradient privacy between clients.

B. Privacy-Preserving Decentralized Federated Learning

There are a significant number of researches in the DFL community addressing the problem of data privacy protection between clients. Nguyen et al. [35] designed a DFL method to ensure data privacy based on knowledge distillation. Chen et al. [36] employed differential privacy to secure data during DFL transmission. Arapakis et al. [37] proposed a privacy-preserving peer-to-peer learning method based on cryptographic primitives (i.e., AHE). Pasquini et al. [38] observed the risk of privacy disclosure arising from network topology and consensus distance for the local model update in decentralized learning, respectively, and proposed two novel membership inference attacks, *local generalization*, and *system knowledge*, in this way. Finally, they give two important strategies to defend against the attack by adopting SA protocols with noise masking and robust aggregation protocols, which certainly fit with the robust SA protocols advocated in this paper.

Unfortunately, the above solutions employ privacy-preserving means with expensive computational costs that make their practicality limited and lack Byzantine robust defense strategies for poisoning attacks.

C. Byzantine-Resilience Strategy

There are a variety of schemes that provide a feasible approach to model poisoning attacks. Median [39] selects the median of the local gradient as the global model update based on the median defense strategy. Trimmed-mean [18] eliminates the largest and smallest values of the local gradients and then averages them out as the new global model to filter out the malicious gradients. Krum [17] employs the Euclidean distance to identify outliers and selects the closest value of the gradients to the neighboring gradient as the global gradient. Auror [40] uses k-means clustering to identify outliers in the local gradients. Sybils [41] utilizes cosine similarity to identify outliers in the nearest gradients. FL-trust [4] identifies outliers based on the strong assumption that the server maintains a pure dataset to validate the cosine similarity between the local gradients and well-gradients generated by training on the pure dataset. This approach contradicts the original intent of PPFL, which strictly prohibits servers from accessing clients' private data and training tasks should be offloaded to the client locally. However, these schemes lack Byzantine-resilience considerations for applicability to heterogeneous data in FL, directly computing local gradients can reveal data privacy and lack security, and filtering outliers based on server coordination is not suitable for DFL architectures. In summary, PPDFL urgently requires a secure and practical Byzantine-resilience defense strategy to resist malicious gradient corruption.

III. Preliminaries

A. Privacy-Preserving Federated Learning

Given a fixed set $\mathcal{M}$ of clients, the server selects a subset $\mathcal{U}(\mathcal{U} \leq \mathcal{M})$ of $\mathcal{M}$ each round, and the local datasets $\mathcal{D}_i$ for each client $i \in \mathcal{U}$ is sampled from the distribution $\mathcal{D}$. The model optimization problem for FL can be formally represented as

$$\arg\min_{\mathbf{w}\in\mathbb{R}^d} \left\{ l(\mathbf{w}) = \frac{1}{\mathcal{M}} \sum_{i=1}^{\mathcal{M}} l_i(\mathbf{w}) \right\} \tag{1}$$

where $l_i(\mathbf{w}) = \mathbb{E}_{\xi\sim\mathcal{D}}[l_i(\mathbf{w};\xi)]$ is the expected loss of client i trained on a random data sample ξ to generate model parameters $\mathbf{w}$, where ξ follows the distribution of $\mathcal{D}$.

Each of the clients $i \in \mathcal{U}$ performs stochastic gradient descent on the private dataset $\mathcal{D}_i$ with the global model $\mathbf{w}^{(0)}$ broadcast by the server to obtain the local gradient $\mathbf{g}_i^{(j)}$ and sends the $\mathbf{g}_i^{(j)}$ to the server. Then the server aggregates the received local gradients $\mathbf{g}_i^{(j)}$ to get the global gradient $\mathbf{g}_o^{(j)} = \frac{1}{|\mathcal{U}|}\sum_{i\in\mathcal{U}} \mathbf{g}_i^{(j)}$, and updates the global model $\mathbf{w}^{(j)} = \mathbf{w}^{(j-1)} - \alpha\mathbf{g}_o^{(j)}$. where α is the learning rate.

While in PPFL, clients are generally required to adopt certain privacy techniques (i.e., homomorphic encryption, differential privacy and secure multi-party computing) to ensure the confidentiality of gradient $\mathbf{g}_i$. In this paper, we generate a random mask $\mathbf{sk}$ with the random seed agreed by the pair-wise clients, padding $\mathbf{sk}$ one-time to $\mathbf{g}_i$ and broadcasting the masked value $[\![\mathbf{g}_i + \mathbf{sk}]\!]$ into $|\mathcal{U}|$ copies to other clients $j, \{i \neq j, j \in \mathcal{U}\}$ using secret sharing.

B. Multi-Secret Sharing

Shamir secret sharing scheme [6] is limited to sharing only a single secret, and each participant's share of the secret will be disclosed after its reconstruction. Moreover, it is necessary to re-generate and distribute the sub-shares when sharing a new secret again, creating additional communication and computational overheads. It obviously doesn't fit the purpose of the foundation model in FL. Fortunately, we describe the definition of multi-secret sharing below.

Let $\mathbb{F}_q$ be a finite field, a (c, t, m, n) multi-secret scheme satisfies that (i). share m values in the finite field $\mathbb{F}_q$ among n users, (ii). any user i $\{i \in \mathcal{U}, |\mathcal{U}| \geq t\}$ can recover all m secret values, and (iii). any user j $\{j \in \mathcal{U}, |\mathcal{U}| \leq c\}$ cannot infer any information about the m secrets. Typically, a multi-secret sharing scheme consists of two parts, MSS.Share() and MSS.Recon(). MSS.Share$(\mathbf{s}, c, t, \mathcal{U}) \rightarrow \{(i, [\mathbf{s}]_i)\}_{i\in\mathcal{U}}$, where $[\mathbf{s}]_i$ is the sub-secret share distributed to user i, $\mathbf{s} = \{\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_m\}_{\mathbf{s}_i\in\mathbb{F}_q^m}$ is a secret vector of size m and a set of $\mathcal{U}$ with n users. MSS.Recon$(\{(i, [\mathbf{s}]_i)\}_{i\in\mathcal{V}}, t) \rightarrow \mathbf{s}$, where any $\mathcal{V} \subseteq \mathcal{U}$ and $|\mathcal{V}| \geq t$. In this paper, the generation of $\mathbf{s}_i$ follows the setting of the previous work [42]. Assume that $\alpha_i, e_i \xleftarrow{\$} \mathbb{F}_q^m$ are randomly pre-selected elements in $\mathbb{F}_q^m$ and known to n users. In secret share distribution, each user i receives multi-secret share $p(\alpha_i)$, where

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS AND DESCRIPTIONS

Notations	Descriptions	Notations	Descriptions
$\mathbb{Z}_q$	Integer field of module q	$\mathbf{w}$	Global model
$\mathbf{g}_o$	Aggregated global gradient	$\mathbf{g}_u$	Local gradient of client u
$[\![\mathbf{g}_i + \mathbf{sk}]\!]$	Masked gradient	$[\mathbf{s}]_u$	Secret share of client u
$s_{u,v}\|\|c_u$	Concatenate client u's mask	$\hat{\mathbf{g}}_{o,u}^{(j)}$	Cross-updates against global model
$\hat{\mathbf{g}}_{u,u}^{(j)}$	Cross-updates against local model	$sign()$	sign statistical function
p_o	Empirical distribution of global model cross-updates	q_u	Empirical distribution of local model cross-updates
$\mathcal{D}_{KL}(p_o\|\|q_u)$	KL-divergence of cross-updates	$\mathcal{D}_{KL}^u$	Masked KL-divergence
φ	Client Byzantine rate	ρ	Client dropout rate

$p(x)$ is a random degree $t-1$ polynomial in the finite field $\mathbb{F}_q$ fulfilling $\mathbf{s}_i = p(e_i)$. Informally, $p(x) = q(x)\prod_{i=1}^{m}(x-e_i) + \sum_{i=1}^{m}\mathbf{s}_i L_i(x)$, $q(x)$ is a completely random degree $t-m-1$ polynomial and $L_i(x)$ is the Lagrange polynomial $\prod_{j\neq i}\frac{(x-e_j)}{(e_i-e_j)}$.

C. The Number Theoretic Transform (NTT)

The NTT algorithm is a privileged version of implementing the discrete Fourier transform with coefficients that are the primitive roots over a finite field [43]. Let $\mathbf{a} = (\mathbf{a}[0], \ldots, \mathbf{a}[n-1]) \in \mathbb{Z}_q^n$ and ω be primitive n-th root of unity in $\mathbb{Z}_q$, implying $\omega^n \equiv 1 \bmod q$. The forward transformation of NTT $\hat{\mathbf{a}} \leftarrow \mathrm{NTT}(\mathbf{a})$ is denoted as $\hat{\mathbf{a}}[i] = \sum_{j=0}^{n-1} a[j](\omega^i)^j \bmod q$ for $i = 0, 1, \ldots, n-1$. The inverse transformation is denoted as $\mathbf{a} \leftarrow \mathrm{INTT}(\hat{\mathbf{a}})$, where $\mathbf{a}[i] = n^{-1}\sum_{j=0}^{n-1}\hat{\mathbf{a}}[j](\omega^{-i})^j$ and the $\mathbf{a} = \mathrm{INTT}(\mathrm{NTT}(\mathbf{a}))$.

D. Diffie-Hellman Key Agreement

In this paper, we adopt the classical Diffie-Hellman key agreement [44] to agree on adjacent node parameters generated. Specifically, we formalize key agreement as a triple $(\mathbf{KA.param}, \mathbf{KA.gen}, \mathbf{KA.agree})$ to make it easier to understand. The algorithm $\mathbf{KA.param}(\lambda) \rightarrow pp$ generates the public security parameters required for the protocol. $\mathbf{KA.gen}(pp) \rightarrow (s_u^{SK}, s_u^{PK})$ enables each client to generate public-private key pairs. $\mathbf{KA.agree}(s_u^{SK}, s_v^{PK}) \rightarrow s_{u,v}$ implements that each client combines his/her private key s_u^{SK} with the public key s_v^{PK} of any client v to obtain the shared key $s_{u,v}$ of u and v.

IV. PROBLEM STATEMENT

A. Notation

Let $\mathbb{Z}_q$ denote the integer field modulo q and $\mathbb{Z}_q^n$ denote the n-dimensional vector space over $\mathbb{Z}_q$. Bold denotes a vector, i.e., $\mathbf{a} \in \mathbb{Z}_q^n$ is a vector with element $a_1, \ldots, a_n \in \mathbb{Z}_q$. Let $a||b$ denote the concatenation of two values (also between vectors), $[\cdot]$ denote the share generated by the secret sharing algorithm. In this paper, the detailed notations are shown in Table I.

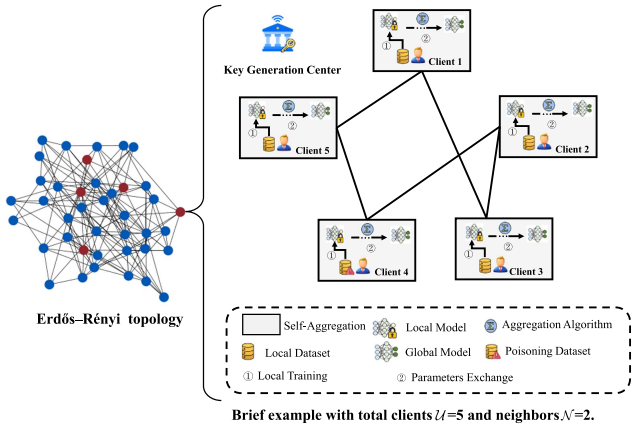


Fig. 1. System Model.

B. Systems Architecture

In contrast to FL, DFL achieves global model aggregation through the distinctive communication mechanism without involving the central aggregation entity. Previous work [3] comprehensively introduced the communication mechanisms in DFL into direct data exchange, peer-to-peer communication and gossip communication, and the above mechanisms complete the gradient transfer by communicating with neighbours. Inspired by randomized communication optimization [8], we generate a communication topology T for PriDFL using an Erdős–Rényi (ER) random graph with k edges. This topology ensures that each client has at least k neighboring nodes and $k \in \{t \leq (1-\varphi-\rho)n \leq 2t\}$, where t denotes the threshold value for secret sharing, n is the total number of clients, and φ and ρ denote the number of Byzantine and dropped clients, respectively.

In order to tolerate client dropouts and the adversary simulating any number of clients to break the protocol, we follow Bonawitz et al. [5] and also require the support of a public key infrastructure that generates a unique client-ID from the client's registered identity and signs the message, which can be verified by the remaining clients to prevent impersonation. In this paper, PriDFL comprises two types of entities, the key generation center (*KGC*) and *Client*, as illustrated in Fig. 1.

- *KGC: KGC* as a trustworthy authority that generates and distributes public-private key pairs for *Client*, while initializing the global model $\mathbf{w}^{(0)}$ communication topology T at the beginning of the protocol execution. From the perspective of real-world applications, selecting a trusted authority to generate security parameters benefits from the authority's high credibility and long-standing social trust. In contrast, choosing a trusted server from competing companies can lead to significant trust issues, and it isn't easy to ensure transparency and correctness when multiple parties are involved. Therefore, the approach of having a trusted authority generate security parameters provides broad social trust and compliance support, which is invaluable in DFL.
- *Client: Client* sorts k neighbors $\mathcal{N}$ by client-ID, and a private authentication channel exists between neighbors in PriDFL. *Client* trains a global model on his/her private

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DETAILED COMPARISONS OF PRIDFL WITH RELATED WORKS, m IS THE VECTOR DIMENSION AND n IS THE NUMBER OF PARTICIPANTS. IN THIS PAPER WE CALCULATE THE OVERHEAD OF **SWIFTAGG+** WITHOUT CATEGORIZING THE NUMBER OF DROPPED AND SEMI-HONEST CLIENTS AS IN THE ORIGINAL ARTICLE, AND UNIFORMLY DENOTE THE NUMBER OF CLIENTS WITH n.

	SecAgg [51]	**SecAgg+** [8]	**FastSecagg** [7]	**BREA** [46]	**SwiftAgg+** [45]	**PriDFL**
Client Computation	$\mathcal{O}(n^2+mn)$	$\mathcal{O}(\log^2 n + m\log n)$	$\mathcal{O}(m\log n)$	$\mathcal{O}(mn\log^2 n + mn^2)$	$\mathcal{O}(n+n\log^2(n))$	$\mathcal{O}(n\log n + m\log n)$
Client Communication	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$	$\mathcal{O}(mn+n)$	$\mathcal{O}(mn)$	$\mathcal{O}(\log^2 n + m)$
Server Computation	$\mathcal{O}(mn^2)$	$\mathcal{O}(n(\log^2 n + m\log n))$	$\mathcal{O}(m\log n)$	$\mathcal{O}((n^2+mn)\log^2 n\log\log n)$	$\mathcal{O}(mn\log^2(n))$	-
Server Communication	$\mathcal{O}(n^2+mn)$	$\mathcal{O}(n(\log n+m))$	$\mathcal{O}(n^2+mn)$	$\mathcal{O}(n^2+mn)$	$\mathcal{O}(mn)$	-

data, agrees on the keys with neighbors, and shares the local masked gradients with neighbors.

C. Security Model

We follow the Byzantine adversary assumption as in [45], [46]. In the Byzantine setting, adversaries can arbitrarily manipulate local training results or poison training data to corrupt local models and hinder global convergence. We also consider a semi-honest setting in which all clients are honest-but-curious: they execute the protocol faithfully to produce gradients while attempting to infer others' data privacy.

We assume a trusted setup phase where a KGC (e.g., a consortium-operated PKI/CA) issues per-client credentials and public parameters needed by PriDFL's secure aggregation. The KGC's role is limited to setup: it binds identities to keys, generates or certifies public parameters, and then exits; it does not participate in any subsequent protocol rounds. In serverless DFL, the KGC may optionally publish a random global initialization once at setup. We explicitly acknowledge that if the KGC is compromised or malicious, confidentiality or authenticity guarantees could degrade; a detailed impact analysis and mitigations are provided in Section VIII.

Finally, we consider clients with low-quality or skewed data and assess their impact on convergence. In the worst case, a client controlled by a Byzantine adversary $\mathcal{A}$ may broadcast poisoned updates to neighbors, significantly degrading global performance.

D. Design Goals

As the challenges mentioned in the introduction, our scheme aims to achieve the following design goals:

- *Computation-communication efficient:* To alleviate the bottleneck of limited computational resources in mobile devices, the proposed protocol should reduce the expensive $\mathcal{O}(n^2)$ exponential computation. Moreover, the entities should achieve linear communication with each other.
- *Byzantine-resilient:* The protocol can diminish the effect of fake local gradients or corrupted gradients, in other words it resists poisoning attacks enabling Byzantine robustness.
- *Dropout-tolerant:* The mechanism should accommodate dynamic client dropouts without interrupting the normal execution of the protocol, ensuring that the confidentiality of the dropped clients' local data is preserved against inference or exposure by remaining participants.
- *Privacy:* The protocol must ensure that clients cannot infer local data from each other, implying no knowledge beyond the aggregated global model.

V. TECHNICAL INTUITION

In this section, we focus on the key components of PriDFL's core components, i.e., NTT-based fast secret sharing, single-masking mechanism, and Byzantine resilient aggregation norms, respectively. At a high level, we guarantee secure data sharing and efficient data synchronization for clients in DFL applications, while supporting lightweight computation and Byzantine robust aggregation. More importantly, clients only learn the aggregated global model without obtaining a little extra private information about other clients. We portray a technical overview of the clients in Fig. 2 and give the detailed implementation process of PriDFL in supplementary materials.

A. Faster-Sharing: NTT Based Secret Sharing

In contrast to Fastshare [7], which is based on the Fast Fourier Transform (FFT), the Fast Number Theoretic Transform (NTT) is isomorphic to the FFT and avoid rounding errors for cyclic convolution of integer sequences [43]. Additionally, NTT is compute-friendly in practical applications based on integer field. The sharing algorithm Faster-sharing$(\mathbf{s}, c, t, \mathcal{U}) \rightarrow \{(u, [\mathbf{s}]_u)\}_{u\in\mathcal{U}}$ is an NTT-based fast secret sharing algorithm that consists of a set of secrets $\mathbf{s} \in \mathbb{Z}_q$ and a set of clients $\mathcal{U}$ of size n as inputs, generating n copies of the shares $[\mathbf{s}]_u \in \mathbb{Z}_q$, with $[\mathbf{s}]_u$ as the share held by each client $u \in \mathcal{U}$.

Let q be prime and let g be a primitive root of $\mathbb{Z}_q$. Fix an NTT length $N \mid (q-1)$ and define the principal N-th root of unity $\omega_N = g^{(q-1)/N} \bmod q$. Let $n = |\mathcal{U}|$ denote the number of clients with $n \le N$ and select n evaluation points x_u from the N-point grid $\{\omega_N^0, \ldots, \omega_N^{N-1}\}$. The multi-secret scheme, on the other hand, mostly adopts the Horner' method in generating shares, which will incur $\mathcal{O}(n^2)$ computational cost and cause an efficiency bottleneck. To cope with the above dilemma, specifically, let $x_1, x_2, \ldots, x_{t+m}, y_1, \ldots, y_n$ be mutually independent values in the finite domain $\mathbb{Z}_q$. In order to split the secrets $\mathbf{s} = \{s_0, s_1, \ldots, s_{m-1}\} \in \mathbb{Z}_q^m$ into n shares $\{[\mathbf{s}]_u\}_{u\in\mathcal{U},|\mathcal{U}|=n}$ and maintain t-privacy and $(t+m)$-reconstruction, we require $n+m \le q, 1 \le t, t+m \le n$. Then, we randomly sample the degree of the $t+m-1$ polynomial $f(x) = \sum_{i=1}^{t+k} f_i x^{i-1}$ satisfying $f(x_i) = s_i$ in the integer domain $\mathbb{Z}_q$, and evaluate the value of the point $y_1, \ldots, y_n$. Note that we need to append the zero coefficients of $f(x)$ to n before evaluating it (i.e., patch zero coefficients from $f(x)$ with degree $t+m-1$ to $f(x)$ with degree $n-1$ since $n \ge t+m$ in general. In this paper, to apply NTT to polynomial evaluation and interpolation, we split $f(x)$ into the sum of an odd function $f_o(x)$ and an even function $f_e(x)$.

$$f(x) = f_e(x^2) + x f_o(x^2) \tag{2}$$

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

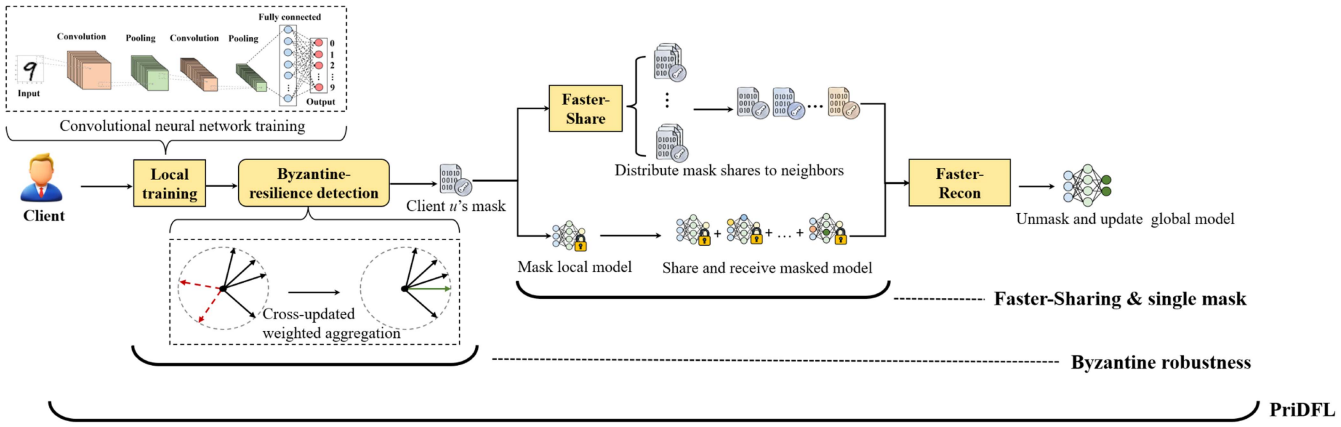


Fig. 2. Technical overview of PriDFL.

Algorithm 1: Faster-sharing.

Procedure *Faster-Share*$(\mathbf{s}, \mathcal{U})$
  Input: integer field $\mathbb{Z}_q$ of size q, the primitive root of unity ω in $\mathbb{Z}_q$, secret $\mathbf{s} \in \mathbb{Z}_q^m$, clients $\mathcal{U}$ of size n (client-IDs), threshold t.
  Output: $\{(u, [\mathbf{s}]_u)\}_{u\in\mathcal{U}}$.
  Random sampling the degree of the $t+m-1$ polynomial $f(x) = \sum_{i=1}^{t+k} f_i x^{i-1}$ satisfying $f(x_i) = s_i$.
  Random sampling values $\mathbf{r} = r_0, \cdots, r_{t-1}, \cdots, r_i \in \mathbb{Z}_q$ such that $\mathbf{s}' = [0] + [\mathbf{s} + \mathbf{r}] \in \mathbb{Z}_q$.
  Perform $\text{INTT}_2()$ to find the coefficients $\mathbf{c}[t+m] = \text{INTT}_2(\mathbf{s}')$.
  Zero-padding $\mathbf{c}$ yields $\mathbf{c}' = \{c_0, c_1, \cdots, c_{t+m}, 0, \cdots, 0_{n-1}\}$.
  Perform $\text{NTT}_3()$ to find vaules $\mathbf{v} = \text{NTT}_3(\mathbf{c}')$.
  Obtain the sub-shares $\{(u, [\mathbf{s}]_u)\}_{u\in\mathcal{U}} \leftarrow \mathbf{v}[1:]$ and share them to the remaining $n-1$ clients.
  return $\{(u, [\mathbf{s}]_u)\}_{u\in\mathcal{U}}$
Procedure *Faster-Recon*$(\{(u, [\mathbf{s}]_u)\}_{u\in\mathcal{U}_o})$
  Input: integer field $\mathbb{Z}_q$ of size q, the primitive root of unity ω in $\mathbb{Z}_q$, online sub-shares with client-IDs $\{(u, [\mathbf{s}]_u)\}_{u\in\mathcal{U}}$, threshold t.
  Output: $\mathbf{s}$.
  case $|\mathcal{U}_o| = n$ **do**
    Perform $\text{INTT}_3()$ to find the coefficients $\mathbf{c}'' = \text{INTT}_3(\{0 + [\mathbf{s}]_u\}_{u\in\mathcal{U}_o, |\mathcal{U}_o|=n})$.
    Perform $\text{NTT}_2()$ to find the values $\mathbf{s}' = \text{NTT}_2(\mathbf{c}''[: t+m+1])$.
    Obtain the final secrets $\mathbf{s} \leftarrow \mathbf{s}'[1 : m+1]$.
  end
  case $t + k \le |\mathcal{U}_o| \le n$ **do**
    Solve for the dropout shares based on NTT's butterfly transformation rule.
    Similarly implemented as in the $|\mathcal{U}_o| = n$ case.
  end
  return $\mathbf{s}$

Moreover, we need to generate the NTT matrices of two different primitive roots of unity in advance, which allows the share generation without sampling points duplicated and avoids the share disclosed secret. Here, we follow the pattern of the research [47] by choosing primes q that satisfy

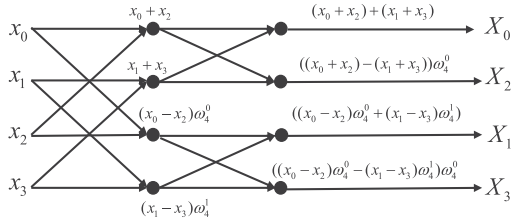


Fig. 3. The 4-node NTT forward transformation with primitive root 2.

$2^a \cdot 3^b \cdot c + 1$, which implies that ω_x and ω_y are 2^a-th and 3^b-th primitive roots of unity, respectively, and let $x_i = \omega_x^i$ and $y_i = \omega_y^j$. Next, in order to find the coefficients of $f(x)$, the zero-padded $\mathbf{s}' = \{0, s_0, s_1, \ldots, s_{m-1}, r_0, \ldots, r_{t-1}\} \in \mathbb{Z}_q$ is backward-transformed by $\text{INTT}_2()$ to find the coefficients $\mathbf{c}$ for the previous $t+m$, where $\mathbf{r} = \{r_0, \ldots, r_{t-1}\}$ is a randomly sampled element of $\mathbb{Z}_q$. Then $\mathbf{c}$ is zero-padded to an n-sized $\mathbf{c}'$, and $\mathbf{c}'$ is forward-transformed by $\text{NTT}_3()$ to find the share $[\mathbf{s}]_u$ and share them to other $n-1$ clients.

In the reconstruction phase, unfortunately, we cannot directly reconstruct the secret using NTT, since it implies a constraint of polynomial degree $n-1$. Instead, the client drop-out dynamically, so the number of online shares $\{[\mathbf{s}]_u\}_{u\in\mathcal{U}_o, |\mathcal{U}_o|\le n}$ in general, where $\mathcal{U}_o$ is the set of surviving clients. Therefore we categorize the reconstruction into two cases: 1). $|\mathcal{U}_o| = n$; 2). $t + k \le |\mathcal{U}_o| < n$.

1) $|\mathcal{U}_o| = n$. The case is relatively straightforward, and we perform the inverse transformation of the share generation. We perform the backward-transformed $\text{INTT}_3()$ on the secret share $\{[\mathbf{s}]_u\}_{u\in\mathcal{U}_o, |\mathcal{U}_o|=n}$ to find the coefficient $\mathbf{c}''$. Then, we pick the previous $t+m+1$ of $\mathbf{c}''$ and perform the forward-transformed $\text{NTT}_2()$ to find $\mathbf{s}'$. And the secret as $\mathbf{s}'[1 : m+1]$.
2) $t + k \le |\mathcal{U}_o| < n$. In this paper we just need to reconstruct once to solve for the final secret $\mathbf{s}$, and the overhead of reconstruction is trivial compared to share generation and sharing. So we construct a system of linear equations to solve for the share of dropout based on the butterfly diagram of NTT, which supports

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

the NTT algorithm. Specifically, we randomly select $\{x_0, x_1, x_2, x_3\} \in \mathbb{Z}_q$, apply the $\mathrm{NTT}_2()$ algorithm to generate $\{X_0, X_1, X_2, X_3\}$, and then distribute $(x_i, X_i), i \in \{0, 1, 2, 3\}$ as a sub-share to $n = 4$ clients. As the computational expression in Fig. 3, we construct the system of linear equations about the dropped shares, we assume the clients that hold x_0 and x_3 are dropout here. Since x_1 and x_2 are known, we can obtain the equations:

$$\begin{cases} ((x_0 - x_2)\omega_4^0 + (x_1 - x_3)\omega_4^1) - X_1 & = 0 \\ ((x_0 + x_2) - (x_1 + x_3))\omega_4^0 - X_2 & = 0 \end{cases} \tag{3}$$

To solve for the above system of equations, we can use Gaussian elimination to derive x_0 and x_3, and then utilize $\mathrm{NTT}_2()$ to derive the corresponding X_0 and X_3. Finally, the n shares are all solved, and we can recover the secret by performing the procedure as in the case of $|\mathcal{U}_o| = n$. Note that this is the case for $n = 4$, of course it generalizes to be correctly computed when n is larger.

According to Algorithm 1, we present the following secret sharing workflow and its corresponding execution phases in PriDFL. Let q be a prime and let g be a primitive root of $\mathbb{Z}_q$. Following the construction in this paper, we choose q such that $q - 1 = 2^a \cdot 3^b \cdot c$, which guarantees the existence of both a 2^a-th and a 3^b-th root of unity. Let ω_x denote a 2^a-th root of unity and let ω_y denote a 3^b-th root of unity; we write $\mathrm{NTT}_2()$ and $\mathrm{NTT}_3()$ for the forward transforms on the ω_x- and ω_y-grids, respectively, and $\mathrm{INTT}_2()$ and $\mathrm{INTT}_3()$ for their inverses. Let $\mathbf{s} = (s_0, \ldots, s_{m-1}) \in \mathbb{Z}_q^m$ be the vector of secrets and let $\mathbf{r} = (r_0, \ldots, r_{t-1}) \in \mathbb{Z}_q^t$ be the randomizer that enforces t-privacy. Let $n = |\mathcal{U}|$ denote the number of clients.

1) *Coefficient synthesis via INTT$_2$():* Form the length-$(t + m)$ frequency sample

$$\mathbf{s}' = \big(0,\, s_0,\, s_1,\, \ldots,\, s_{m-1},\, r_0,\, \ldots,\, r_{t-1}\big) \in \mathbb{Z}_q^{t+m}.$$

Compute the coefficient vector on the ω_x-grid

$$\mathbf{c} = \mathrm{INTT}_2(\mathbf{s}') \in \mathbb{Z}_q^{t+m},$$

which materializes $f(x) = \sum_{j=0}^{t+m-1} c_j x^j$. Because the first $t + m$ spectral slots encode $\mathbf{s}$ and $\mathbf{r}$, any subsequent point evaluation inherits t-privacy.

2) *Batched evaluation via zero-padding and NTT$_3$():* Zero-pad $\mathbf{c}$ to length n to obtain $\mathbf{c}' = (c_0, \ldots, c_{t+m-1}, 0, \ldots, 0) \in \mathbb{Z}_q^n$ and evaluate on the ω_y-grid in a single transform

$$\mathbf{v} = \mathrm{NTT}_3(\mathbf{c}') = (v_1, \ldots, v_n) \in \mathbb{Z}_q^n.$$

Assign the sub-share of client u as $[\mathbf{s}]_u := v_u$. Steps 1–2 replace per-point Horner evaluation of multi-secrets in $\mathcal{O}(nm)$ time by two structured transforms in $\mathcal{O}\big((t+m)\log(t+m) + n \log n\big)$ time.

3) *Reconstruction with all clients online: INTT$_3$() then NTT$_2$():* When $|\mathcal{U}_o| = n$, first invert the ω_y-grid

$$\mathbf{c}'' = \mathrm{INTT}_3\big((0 + [\mathbf{s}]_u)_{u \in \mathcal{U}_o}\big),$$

then take the first $t+m$ coefficients and transform back to the ω_x-grid

$$\mathbf{s}' = \mathrm{NTT}_2\big(\mathbf{c}''[0{:}t{+}m]\big),$$

and finally recover $\mathbf{s} = \mathbf{s}'[1{:}m{+}1]$. Standard normalization by the modular inverses of the transform lengths is applied to the inverse transforms.

4) *Dropout-robust reconstruction as erasure recovery on the NTT$_3$() grid:* When $t+k \le |\mathcal{U}_o| < n$, treat the missing sub-shares as erasures on the ω_y-grid. Recover the erased entries by solving the small linear system induced by the corresponding $\mathrm{NTT}_3()$ submatrix (a Toeplitz–circulant structure that admits fast multiplication), or equivalently by erasure decoding on the transform grid; then proceed as in the full-online case by applying $\mathrm{INTT}_3()$ followed by truncation and $\mathrm{NTT}_2()$. The additional cost depends only on the number of erasures and remains negligible compared with the two global transforms.

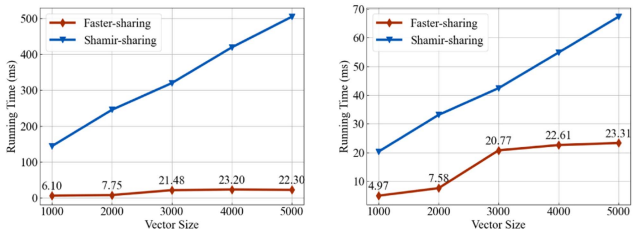
In the full process of supplementary materials for PriDFL, Round 1 (**Sharing**) invokes Steps 3-4 to produce and exchange sub-shares for the concatenated values needed by secure aggregation, while Round 3 (**Unmasking**) invokes Step 4 (falling back to Step 3 when all sub-shares are available) to reconstruct masked sums and remove blinding factors before the final aggregation.

Most of the existing SA schemes implemented with Shamir secret-sharing. We implement both *Faster-sharing* and the Shamir baseline over the prime field $\mathbb{Z}_q$ with $q = 998{,}244{,}353$, which satisfies $q - 1 = 2^{23} \cdot 119$. We fix a primitive root $g = 3$ and derive the N-point principal root of unity $\omega_N = g^{(q-1)/N}$ for any N that divides 2^{23}. In all experiments we use block length $N = 4096$ and process model vectors in contiguous blocks, and we precompute the forward and inverse twiddle tables for this N. For the Shamir baseline, we follow the typical construction over $\mathbb{Z}_q$, we set the number of shares to $|\mathcal{U}|$ and the reconstruction threshold to $t = \lceil (1 - \rho)\,|\mathcal{U}| \rceil$ for a target dropout rate ρ, we choose evaluation points $x_i = i + 1 \bmod q$ for $i \in \{1, \ldots, |\mathcal{U}|\}$, we derive polynomial coefficients by expanding a per-client seed with SHA-256 so that the coefficient randomness and reproducibility remain consistent across methods, we evaluate shares with vectorized Horner's rule, and we reconstruct with standard Lagrange interpolation in $\mathbb{Z}_q$. For *Faster-sharing*, we keep the same field $\mathbb{Z}_q$, the same pair $(|\mathcal{U}|, t)$, the same seed-expansion procedure based on SHA-256, and the same serialization and thread settings, while we accelerate share evaluation and reconstruction through NTT-based batched polynomial operations. Specifically, for each block we compute a forward N-point NTT using ω_N, we perform the pointwise multiplications required by the batched evaluations, and we complete the shares by one inverse N-point NTT per block with the standard normalization factor $N^{-1} \bmod q$. As demonstrated in Fig. 4, our proposed *Faster-sharing* is more effective in handling models with massive parameters and high-dimensional vectors, which is especially emphasized in the later experimental analysis.

B. Lightweight Single Mask

Masking with one-time padding technique because of its user-friendly and perfect secrecy to compute private values [5], [27], [28], [29]. In this paper, we also adopt it to mask client inputs,

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.



(a) Sharing (b) Reconstruct

Fig. 4. The efficiency of sharing and reconstruction between Faster-sharing and Shamir.

however, we focus on decentralized federated learning, which makes secret masking and noise cancellation different from previous researches. Specifically, all clients jointly complete federated training under the k-edge ER communication architecture, and each client $u, u \in \mathcal{U}$ has a neighbor set of $\mathcal{N}_u$. Suppose that client u inputs is $\mathbf{x}_u$, and $s_{u,v}$ and $s_{v,u}$ are pseudo-random seeds agreed upon for $u > v, v \in \mathcal{N}_u$ and $u < v$, respectively. If u subtracts the random vector $\mathbf{s}_{u,v} = \mathbf{PRG}(s_{u,v})$ and adds the vector $\mathbf{s}_{v,u} = \mathbf{PRG}(s_{v,u})$, then the mask will be cancelled when their vectors are summed, but the client's private inputs will not be revealed. In other words, each client u computes:

$$\mathbf{y}_u = \mathbf{x}_u - \mathbf{s}_{u,v} + \mathbf{s}_{v,u} \tag{4}$$

Share $\mathbf{y}_u$ on a secure channel to neighbor $\mathcal{N}_u$, each client computes:

$$\begin{aligned}\mathbf{z} &= \sum_{u\in\mathcal{U}}^{|\mathcal{U}|} \mathbf{y}_u \\ &= \sum_{u\in\mathcal{U}}^{|\mathcal{U}|} \left(\mathbf{x}_u - \sum_{v\in\mathcal{N}_u, u>v} \mathbf{s}_{u,v} + \sum_{v\in\mathcal{N}_u, u<v} \mathbf{s}_{v,u} \right) \\ &= \sum_{u\in\mathcal{U}}^{|\mathcal{U}|} \mathbf{x}_u \bmod \mathbb{Z}_q \end{aligned} \tag{5}$$

To tolerate client dropping out and failing to complete the protocol making the aggregation unable to eliminate its noise, we utilize a threshold multi-secret sharing scheme, *Faster-sharing*, to share the pseudorandom generator seeds $\mathbf{s}_{u,v}, \mathbf{s}_{v,u}$ agreed by each client to the other clients. This allows a sufficient number of parties ($\geq t$) surviving to recover the keys of the dropped clients.

To facilitate understanding of single mask applications, we provide three client operation examples. Let $\mathcal{U} = \{1, 2, 3\}$ and assume a complete neighbor graph so that $\mathcal{N}_u = \mathcal{U} \setminus \{u\}$. For each ordered pair (u, v), users u and v run a key agreement to derive a seed $s_{u,v}$ and define a mask vector $\mathbf{s}_{u,v} = \mathbf{PRG}(s_{u,v}) \in \mathbb{Z}_q$. Client u forms its masked message by

$$\mathbf{y}_u = \mathbf{x}_u - \sum_{v\in\mathcal{N}_u, u>v} \mathbf{s}_{u,v} + \sum_{v\in\mathcal{N}_u, u<v} \mathbf{s}_{v,u}.$$

Explicitly, the three users send

$$\mathbf{y}_1 = \mathbf{x}_1 + \mathbf{s}_{1,2} + \mathbf{s}_{1,3},$$

$$\mathbf{y}_2 = \mathbf{x}_2 - \mathbf{s}_{2,1} + \mathbf{s}_{2,3},$$

$$\mathbf{y}_3 = \mathbf{x}_3 - \mathbf{s}_{3,1} - \mathbf{s}_{3,2}.$$

Summing all contributions gives pairwise cancellation on every edge:

$$\mathbf{y}_1 + \mathbf{y}_2 + \mathbf{y}_3 = \mathbf{x}_1 + \mathbf{x}_2 + \mathbf{x}_3 \pmod{\mathbb{Z}_q}.$$

During the sharing round, each client u secret-shares (with threshold t) the seeds whose corresponding masks appear with a "+" sign in $\mathbf{y}_u$, i.e., $\{s_{v,u} : v \in \mathcal{N}_u, v > u\}$, using the NTT-based *Faster-sharing* so that any set of $\geq t$ survivors can reconstruct them. Suppose client 1 drops before sending $\mathbf{y}_1$. The surviving users 2 and 3 compute

$$\mathbf{y}_2 + \mathbf{y}_3 = \mathbf{x}_2 + \mathbf{x}_3 - \mathbf{s}_{2,1} - \mathbf{s}_{3,1}.$$

Since $s_{1,2}$ and $s_{1,3}$ were shared by client 1in the offline phase, users 2 and 3 (or any $\geq t$ survivors) reconstruct these seeds and locally regenerate $\mathbf{s}_{1,2}, \mathbf{s}_{1,3}$. Adding them back cancels the outstanding masks and reveals the desired sum of surviving models:

$$\begin{aligned}(\mathbf{y}_2 + \mathbf{y}_3) &= \mathbf{x}_2 + \mathbf{x}_3 - \mathbf{s}_{2,1} - \mathbf{s}_{3,1} + \mathbf{s}_{1,2} + \mathbf{s}_{1,3} \pmod{\mathbb{Z}_q} \\ &= \mathbf{x}_2 + \mathbf{x}_3.\end{aligned}$$

This example illustrates that PriDFL's single-mask rule guarantees exact pairwise cancellation when all users are online, while the *Faster-sharing* of the positively signed seeds enables threshold recovery of the missing counterpart masks under dropouts, preserving privacy and correctness in the decentralized topology.

C. Byzantine-Resilience Aggregation

To prevent malicious gradients from corrupting the aggregation results, we design a Byzantium-resilience aggregation mechanism as follows:

Decentralized parallel stochastic gradient descent algorithm is usually used to solve the model convergence problem in DFL [48]. And it can be observed that the adjoining node weights, as an important part of the gradient averaging, are picked to help the model converge quickly. Independently, SignGuard [49] statistically characterises the gradient signs and computes the update magnitude of the local gradient, overcoming the drawbacks of distance- and median-based methods that are not resistant to specific attacks [50], while enabling robust aggregation without relying on other external information.

Inspired by these observations, we design a weight calculating scheme based on the cross-update gradient, which makes the client Byzantine robust for local aggregation. Specifically, client $u \in \mathcal{U}$ calculates the cross-update gradients $\hat{\mathbf{g}}_{o,u}^{(j)} = \mathbf{g}_u^{(j)} - \mathbf{g}_o^{(j-1)}$ and $\hat{\mathbf{g}}_{u,u}^{(j)} = \mathbf{g}_u^{(j)} - \mathbf{g}_u^{(j-1)}$. It then computes the empirical sign distributions of $\hat{\mathbf{g}}_{o,u}^{(j)}$ and $\hat{\mathbf{g}}_{u,u}^{(j)}$ to obtain $\delta_{o,u}^{(j)} = sign(\hat{\mathbf{g}}_{o,u}^{(j)})$ and $\delta_{u,u}^{(j)} = sign(\hat{\mathbf{g}}_{u,u}^{(j)})$, where $\mathbf{g}_{o,u}^{(j)}$ is the global gradient of round j, $\mathbf{g}_{u,u}^{(j)}$ is the local gradient of client u in round j and $sign(\cdot)$ represents a function that computes the proportions of positive, negative, and zero elements in a gradient vector. Instead, KL-divergence quantifies the information difference between the two distributions satisfactorily, we measure the distance between the empirical sign distributions $\delta_{o,u}^{(j)}$ (conveniently denoted as p_o below) and $\delta_{u,u}^{(j)}$ (denoted by q_u below) of the

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

cross-update vector with the KL-divergence as follows:

$$\mathcal{D}_{KL}(p_o||q_u) = \sum_{x=1,0,-1} p_o(x) \log\left(\frac{p_o(x)}{q_u(x)}\right) \tag{6}$$

where x is the value of $\hat{\mathbf{g}}_{o,u}^{(j)}$ and $\hat{\mathbf{g}}_{u,u}^{(j)}$ at positive, zero and negative elemental signs. Compute the ratio of KL-divergence for client u as follows:

$$\theta_u \leftarrow \frac{\mathcal{D}_{KL}(p_o||q_u)}{\mathcal{D}_{sum}} \tag{7}$$

where θ_u is the weight of client u, it also implies that cross-update gradients with higher KL-divergence will contribute less to the local model update.

In PriDFL, each client computes cross-update vectors with respect to both the previous global model and its own previous local model, and then derives the empirical sign distributions for these two cross-updates. The client measures their discrepancy by the KL divergence and obtains a normalized weight by dividing the KL value by the private sum of all clients' KL values, after which the weighted aggregation is performed locally under the single-mask mechanism. This design realizes a purely local, privacy-preserving validity check on the directional correctness of each update before aggregation, and it relies only on sign distributions rather than raw gradients or inter-client exchanges, which conforms to our decentralized and privacy-preserving constraints.

To address the concern that KL divergence can be sensitive when some sign categories have very small empirical counts, we implement three stabilization steps at the client side before masking and broadcasting the privatized KL terms. First, we apply additive smoothing to the three-category sign histogram so that each category probability is lower bounded by a small constant, which prevents undefined logs and large spikes. Second, we compute the KL value on an exponential moving average of recent-round sign distributions rather than on a single round, which reduces variance induced by stochastic mini-batches. Third, we clip the resulting per-client KL value to a fixed interval and renormalize the weights by the private sum, which ensures bounded influence. These operations are performed locally before the masked KL values are summed via secret-sharing, and they do not change PriDFL's privacy guarantees.

Remarkably, KL-divergence is an essential metric for the global model contribution that cannot be transmitted publicly. We trickily sum the KL-divergence of all clients securely via the homomorphic additivity of *Faster-sharing*. More specifically, each client samples a random element $c_u \leftarrow \mathbb{Z}_q$ to blind the KL-divergence, and then broadcast the masked KL-divergence $\mathcal{D}_{KL}^u$ to the remaining clients. In the **Unmasking** round, the client reconstructs the masked KL-divergence of all clients based on the homomorphic additivity and finally eliminates the blinding element $\sum_{v\in\mathcal{U}_3} c_v$ to derive the sum of KL-divergence $\mathcal{D}_{sum}$. All the above processes are performed locally at the clients and do not disclose the weight information of the clients.

VI. Theoretical Analysis

A. Security Analysis

In this section, we demonstrate the security of our protocol in the honest but curious setting. In detail, we prove that the joint view of PriDFL between any less than threshold t clients does not disclose the input information of others except for knowing the aggregation result. Our protocol runs with the security parameter λ required by the cryptographic primitive, the set $\mathcal{U}$ of n clients, and the threshold t.

$\mathcal{U}_i$ denotes the number of clients in each round of the protocol, and $\mathbf{x}_{\mathcal{U}'} = \{\mathbf{x}_u\}_{u\in\mathcal{U}'}$ denotes the input $\mathbf{x}_u$ of any subset of clients $\mathcal{U}' \subseteq \mathcal{U}$. We prove our protocol with a hybrid argument approach in secure multi-party computation, where the view consists of the security parameter λ, the internal state of the party and all messages received from other parties in the protocol. Given any subset $\mathcal{C} \subseteq \mathcal{U}$ of parties, let $\text{REAL}_{\mathcal{C}}^{\mathcal{U},t,\lambda}(\mathbf{x}_{\mathcal{U}}, \mathcal{U}_1, \mathcal{U}_2, \mathcal{U}_3)$ denote the random variable of the combined views of the parties in $\mathcal{C}$ during the execution of the protocol.

We consider security between honest but curious clients. In particular, in our ring structure, a client combines its own knowledge with the information received from neighboring clients. Any client in the protocol can simulate by combining the information held and received. Moreover, these clients learn nothing except their own inputs and aggregated results. Additionally, there must be fewer than t honest but curious clients in the simulation view otherwise our protocol is not secure.

We list the cryptographic assumptions used by PriDFL and reference them in the analysis.

- A1 *Decisional Diffie–Hellman (DDH):* Let $\mathbb{G}$ be a cyclic group of prime order q with generator g. For uniformly random $a, b, c \leftarrow \mathbb{Z}_q$, the distributions (g^a, g^b, g^{ab}) and (g^a, g^b, g^c) are computationally indistinguishable. This assumption justifies the indistinguishability of Diffie-Hellman-derived shared secrets from uniform and supports the security of the key-agreement step used to derive pairwise pads and session keys in PriDFL.
- A2 *Pseudorandomness:* The pseudorandom generator used to derive masking pads from client seeds outputs bitstrings that are computationally indistinguishable from uniform, which underlies the privacy of masked updates and the blinding of per-client statistics before aggregation.
- A3 *IND-CPA security of public-key encryption:* The public-key encryption used in the setup phase to deliver per-client seeds or session keys to authenticated recipients achieves indistinguishability under chosen-plaintext attacks, which ensures confidentiality of key material distributed during initialization.
- A4 *Hiding of additive secret sharing:* The additive sharing used for masked-sum aggregation hides each client's value provided that masks or shares are uniform and independent and that only the final sum is opened; when masks are derived from a PRG, the hiding is computational and reduces to A1. This assumption underlies the privacy of local KL statistics and gradient-sign aggregates.

Theorem 1: There exists a polynomial probability time (PPT) simulator SIM for all $\lambda, t, \mathcal{U}, \mathbf{x}_{\mathcal{U}}, \mathcal{U}_1, \mathcal{U}_2, \mathcal{U}_3$ and any subset $\mathcal{C} \subseteq \mathcal{U}, |\mathcal{C}| \leq t$, clients $\mathcal{U}_3 \subseteq \mathcal{U}_2 \subseteq \mathcal{U}_1 \subseteq \mathcal{U}$. The output of SIM and the output of $\text{REAL}_{\mathcal{C}}^{\mathcal{U},t,k}$ are computationally

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

indistinguishable (CI).

$$\text{REAL}_{\mathcal{C}}^{\mathcal{U},t,\lambda}(\mathbf{x}_{\mathcal{U}},\mathcal{U}_1,\mathcal{U}_2,\mathcal{U}_3) \approx_{CI} \text{SIM}_{\mathcal{C}}^{\mathcal{U},t,\lambda}(\mathbf{x}_{\mathcal{C}},\mathbf{z},\mathcal{U}_1,\mathcal{U}_2,\mathcal{U}_3)$$

where

$$\mathbf{z} = \begin{cases} \sum_{u\in\mathcal{U}_2\setminus\mathcal{C}} \mathbf{x}_u & \text{if } |\mathcal{U}_2| \geq t \\ \perp & \text{otherwise.} \end{cases}$$

Proof: We define the simulator SIM with a series of experiments describing the random variable REAL and computationally indistinguishable between adjacent experiments.

- $\mathbf{Hyb}_0$: This random variable has exactly the same distribution as REAL, which is the joint view of the parties $\mathcal{C}$ at the real execution of the protocol.
- $\mathbf{Hyb}_1$: In this hybrid, we fix the operation of the client in $\mathcal{U}_2 \setminus \mathcal{C}$. The simulator substitutes the keys $s_{u,v}$ agreed between the original client u and the neighbors $v \in \mathcal{N}_u$ with a uniformly random element $s'_{u,v}$. The DDH assumption **A1** ensures that this hybrid is indistinguishable from the previous one.
- $\mathbf{Hyb}_2$: In this hybrid, we substitute the transmitted shares $[\mathbf{sk}_u]_v$ in client $\mathcal{U}_2 \setminus \mathcal{C}$ with a 0-padding vector. The security of *Faster-sharing* and assumption **A4** guarantees that the shares of 0 and $\mathbf{sk}_u$ have the same distribution, and this hybrid is indistinguishable from the previous one.
- $\mathbf{Hyb}_3$: In this hybrid, we send messages using a secure channel with AES symmetric encryption. Initially, we replace the encrypted ciphertexts of all honest party u' in the set $\mathcal{U}_2 \setminus \mathcal{C}$ and then send the ciphertexts of all zeros to the other honest parties without sharing $\mathbf{sk}_u$ and $\mathcal{D}_{KL}^u$ according to the protocol. And the honest clients in that set continue to respond to the correct share of $\mathbf{sk}_u$ and $\mathcal{D}_{KL}^u$ in the **Unmasking** round. Only the ciphertexts has changed in this hybrid, and the **A3** assumption ensures that it remains indistinguishable from the previous hybrid.
- $\mathbf{Hyb}_4$: The distribution of this hybrid is exactly identical to the previous one, since we have previously replaced the all shares of $\mathbf{sk}_u$ in the set $u' \in \mathcal{U}_3 \setminus \mathcal{C}$, and the other parties can only get 0 shares. In the hybrid model, the adversary cannot receive any additional shares of $\mathbf{sk}_u$ in set $u \in \mathcal{U}_3 \setminus \mathcal{C}$ during **Unmasking** round. So the joint view of $M_{\mathcal{C}}$ holds only $|\mathcal{C}| \leq t$ shares of $\mathbf{sk}_u$, when $\mathbf{z} = \perp$. This can be guaranteed by the property of *Faster-sharing* that the distribution of any share of 0 is the same as the distribution of any share of $\mathbf{sk}_u$, while assumption **A4** guarantees that the distribution is identical to the previous hybrid model. Note that the *Faster-sharing* privacy guarantee is similar to [7].
- $\mathbf{Hyb}_5$: In this hybrid, we substitute the output of the **PRG** performed by all parties in $u' \in \mathcal{U}_3 \setminus \mathcal{C}$ with uniformly randomized vectors. Note that SIM sends the value $\mathbf{y}_u$ to the neighbors $v \in \mathcal{N}_{u'}$ replacing it with $\mathbf{y}_{u'} \leftarrow \mathbf{x}_u + \mathbf{PRG}(s'_{u,v})$. All parties in $u' \in \mathcal{U}_3 \setminus \mathcal{C}$ do not hold any additional shares of $\mathbf{sk}$ so that the generation of random values does not depend on the **PRG** seed. We utilize the randomness of the **PRG** (i.e., assumption **A2**) to ensure that this hybrid is indistinguishable from the previous one.
- $\mathbf{Hyb}_6$: We can replace the mask gradients $\mathbf{y}_u = \mathbf{x}_u - \sum_{v\in\mathcal{N}_u, u>v} \mathbf{s}_{u,v} + \sum_{v\in\mathcal{N}_u, u<v} \mathbf{s}_{v,u}$ of all parties in $u \in \mathcal{U}_3 \setminus \mathcal{C}$ at the **Masking** round with $\mathbf{y}_u = \mathbf{r}_u - \sum_{v\in\mathcal{N}_u, u>v} \mathbf{s}_{u,v} + \sum_{v\in\mathcal{N}_u, u<v} \mathbf{s}_{v,u}$ at this time. Since the generated masks $\mathbf{sk}_u$ have been previously replaced with uniformly random vectors $\mathbf{PRG}(s'_{u,v})$, $\mathbf{x}_u - \sum_{v\in\mathcal{N}_u, u>v} \mathbf{s}_{u,v} + \sum_{v\in\mathcal{N}_u, u<v} \mathbf{s}_{v,u}$ is uniformly random as is $\mathbf{r}_u - \sum_{v\in\mathcal{N}_u, u>v} \mathbf{s}_{u,v} + \sum_{v\in\mathcal{N}_u, u<v} \mathbf{s}_{v,u}$, where $\mathbf{r}_u$ is a randomly sample vector of clients $u \in \mathcal{U}_3 \setminus \mathcal{C}$ that satisfies $\sum_{u\in\mathcal{U}_3\setminus\mathcal{C}} \mathbf{r}_u = \sum_{u\in\mathcal{U}_3\setminus\mathcal{C}} \mathbf{x}_u$. Thus $\frac{1}{|\mathcal{U}_3\setminus\mathcal{C}|} \sum_{u\in\mathcal{U}_3\setminus\mathcal{C}} \theta_u \cdot \mathbf{r}_u = \mathbf{z}$, so this hybrid is identically distributed as the previous one.

  In case $u \in \mathcal{U}_3 \setminus \mathcal{C}$, the simulator SIM can already simulate REAL without knowing $\mathbf{x}_u$ of any set $u \notin \mathcal{C}$, hence we can ignore the other hybrid models that follow.

The above hybrids prove that the output of the PPT simulator SIM in the ideal environment is computationally indistinguishable from the output of REAL in the real environment. Therefore, PriDFL guarantees data security.

B. Performance Analysis

Computational cost: We assume that $k = \mathcal{O}(\log n)$ i.e., the complexity of selecting k neighbors in an ER random topology. The client computational cost is mainly divided into (1) performing k key agreements for $\mathcal{O}(n \log n)$ times, (2) generating secret shares takes $\mathcal{O}(n \log n)$ times, and (3) generating $\mathbf{s}_{u,v}$ for the parameter masks called $\mathbf{PRG}$ seeds costs $\mathcal{O}(2m \log n)$. Overall, the computational cost per client is $\mathcal{O}(n \log n + m \log n)$.

Communication overhead: The communication overhead is broken down into three main parts: (1) exchanging public keys $3k$ times between clients, and (2) transferring masked secret shares of $2k$ times, and sending a masked data vector of size m to the neighbors (3) broadcasting the masked KL-divergence $\mathcal{O}(n-1)$. The total communication complexity is $\mathcal{O}(\log^2 n + m)$. The computational/communication overhead of PriDFL compared to the baseline is shown in Table II.

C. Correctness Analysis

We show that the single-mask mechanism achieves exact cancellation in the fully online case and that, under dropouts, the aggregate of the surviving inputs can still be recovered when at least t authenticated clients remain.

Theorem 2: Let $\mathcal{U}$ be the client set and $\mathcal{U}_o \subseteq \mathcal{U}$ denote the online clients in a round. For each unordered pair $\{u, v\}$ with $u < v$, let the two users agree on a seed $s_{u,v}$ and define the mask vector $\mathbf{s}_{u,v} = \mathbf{PRG}(s_{u,v}) \in \mathbb{Z}_q$. Each client u sends the masked vector

$$\mathbf{y}_u = \mathbf{x}_u - \sum_{v\in\mathcal{N}_u,\, v<u} \mathbf{s}_{u,v} + \sum_{v\in\mathcal{N}_u,\, v>u} \mathbf{s}_{v,u}. \tag{8}$$

Assume that, in the sharing phase, client u secret-shares (with threshold t) all seeds $\{ s_{u,v} : v > u \}$ appearing in (8). Then:

1) If $\mathcal{U}_o = \mathcal{U}$, it holds that $\sum_{u\in\mathcal{U}} \mathbf{y}_u = \sum_{u\in\mathcal{U}} \mathbf{x}_u \pmod{\mathbb{Z}_q}$.
2) If $\mathcal{U}_o \subset \mathcal{U}$ with $|\mathcal{U}_o| \geq t$, then the surviving clients can reconstruct the aggregate of the online inputs by locally

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

adding the masks corresponding to the dropped neighbors:

$$\left(\sum_{u\in\mathcal{U}_o}\mathbf{y}_u\right)+\sum_{\substack{u\in\mathcal{U}_o\\ v\in\mathcal{U}\setminus\mathcal{U}_o,\,v<u}}\mathbf{s}_{u,v}-\sum_{\substack{u\in\mathcal{U}_o\\ v\in\mathcal{U}\setminus\mathcal{U}_o,\,v>u}}\mathbf{s}_{v,u}=\sum_{u\in\mathcal{U}_o}\mathbf{x}_u\pmod{\mathbb{Z}_q},\tag{9}$$

where each $\mathbf{s}_{v,u}$ (with $v>u$) is regenerated from the threshold reconstruction of $s_{v,u}$ via Faster-Recon.

Proof: (1) All clients online. By expanding (8) and summing over $u\in\mathcal{U}$, every mask $\mathbf{s}_{u,v}$ with $u<v$ appears exactly once with a "$-$" in $\mathbf{y}_u$ and once with a "$+$" in $\mathbf{y}_v$, hence pairwise cancellation yields

$$\sum_{u\in\mathcal{U}}\mathbf{y}_u=\sum_{u\in\mathcal{U}}\mathbf{x}_u\pmod{\mathbb{Z}_q}.$$

(2) Dropouts. Let $\mathcal{R}=\mathcal{U}\setminus\mathcal{U}_o$ be the dropped clients. Summing (8) over $u\in\mathcal{U}_o$ gives

$$\sum_{u\in\mathcal{U}_o}\mathbf{y}_u=\sum_{u\in\mathcal{U}_o}\mathbf{x}_u-\sum_{\substack{u\in\mathcal{U}_o\\ v\in\mathcal{N}_u,\,v<u}}\mathbf{s}_{u,v}+\sum_{\substack{u\in\mathcal{U}_o\\ v\in\mathcal{N}_u,\,v>u}}\mathbf{s}_{v,u}.$$

Masks along edges internal to $\mathcal{U}_o$ still cancel pairwise as in case (1). The only remaining terms lie on the cut between $\mathcal{U}_o$ and $\mathcal{R}$:

$$\sum_{u\in\mathcal{U}_o}\mathbf{y}_u=\sum_{u\in\mathcal{U}_o}\mathbf{x}_u-\sum_{\substack{u\in\mathcal{U}_o,\,v\in\mathcal{R}\\ v<u}}\mathbf{s}_{u,v}+\sum_{\substack{u\in\mathcal{U}_o,\,v\in\mathcal{R}\\ v>u}}\mathbf{s}_{v,u}.$$

To cancel these cut-edge masks, survivors follow rule (9) which yields

$$\left(\sum_{u\in\mathcal{U}_o}\mathbf{y}_u\right)+\sum_{\substack{u\in\mathcal{U}_o,\,v\in\mathcal{R}\\ v<u}}\mathbf{s}_{u,v}-\sum_{\substack{u\in\mathcal{U}_o,\,v\in\mathcal{R}\\ v>u}}\mathbf{s}_{v,u}=\sum_{u\in\mathcal{U}_o}\mathbf{x}_u\pmod{\mathbb{Z}_q}.\tag{10}$$

When the needed seed is owned by a dropped client (e.g., $\mathbf{s}_{v,u}$ with $v>u$), survivors reconstruct $s_{v,u}$ via *Faster-Recon* and regenerate $\mathbf{s}_{v,u}=\mathbf{PRG}(s_{v,u})$ to apply the subtraction above; when it is owned by a survivor (e.g., $\mathbf{s}_{u,v}$ with $v<u$), it is computed locally and added back. All equalities are over $\mathbb{Z}_q$. □

The theorem states that the single-mask rule guarantees exact pairwise cancellation when all users are online. At the same time, the *Faster-sharing* of seeds with positive sign ensures that any set of at least t authenticated survivors can reconstruct the missing counterpart masks and, therefore, recover the aggregate of the surviving inputs in the decentralized topology.

VII. Performance Evaluation

We evaluate our PriDFL multi-dimensionally and compare it with typical solutions from three aspects: classification accuracy, computational efficiency and Byzantine-resilience, respectively. In addition, the PriDFL implementation is described at https://github.com/SafeAI-Wshuai/PriDFL.

A. Experimental Setup

1) *Datasets:* Experimental evaluation datasets are mainly MNIST and CIFAR-10, which are the image classification datasets mostly used for analysis in PPFL. We follow the setup of previous work where each client holds $1/|\mathcal{U}|$ random slices of the total dataset and $|\mathcal{U}|$ is the total number of clients, where $|\mathcal{U}|=100$ and randomly selected 10% of the clients to participate in model aggregation in this paper [51].
MNIST: MNIST [52] is a 10-class digit picture classification dataset, consisting of 60,000 training samples and 10,000 test samples.
CIFAR-10: CIFAR-10 [53] is a color image consisting of 50,000 training samples and 10,000 test samples that include images of airplanes, cat, dog.

TABLE III
Unified Hyperparameters and Protocol Parameters

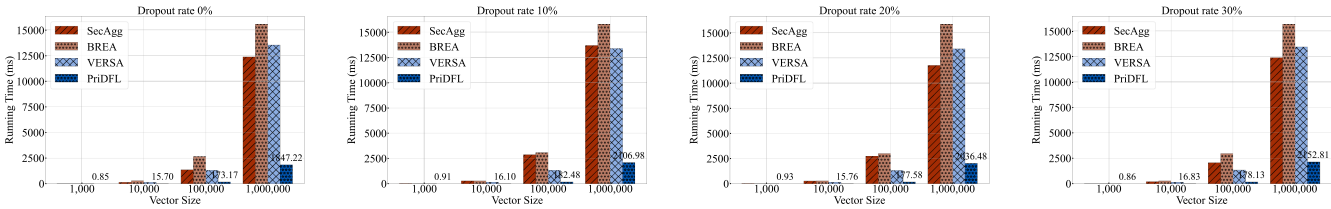
Hyperparameter	Value	Meaning
e	3	local iterations
b	128	batch size
η	0.007	initial learning rate
ξ	0.9	learning rate decay
ζ	5e-4	weight decay
μ	0.9	momentum
γ	$\{5,6,8,10\}$	truncation factor
$\|\mathcal{U}\|$	$\{100,200,300,400,500\}$	clients
ρ	$\{0\%,10\%,20\%,30\%\}$	dropout rate
q	998,244,353 with $q-1=2^{23}\cdot 119$	prime field
g	3	primitive root
N	4096	NTT block length
t	$\lceil(1-\rho)\,\|\mathcal{U}\|\rceil$	secret sharing threshold

2) *Data Settings:* In the implementation, we evaluate PriDFL in the non-IID setting. We follow the previous work [46] to slice the training samples into 100 sub-datasets.
3) *Model Parameter Settings:* We locally trained LeNet5 model on MNIST and ResNet18 model on CIFAR-10, making PriDFL to handle large-scale parameters models perfectly. LeNet-5 and ResNet-18 detailed model structure information is referenced from [54], [55], where the local iterations are 3, the batch size is 128, and the learning rate is 0.007. Detailed hyperparameter settings are shown in Table III.
4) *Data Truncation:* Since *Faster-sharing* works over a finite field $\mathbb{Z}_q$, we preprocess floating point numbers to integers before secret sharing. Given a large rounding factor γ convert a floating point number x to an integer $x'\leftarrow\mathbf{round}(x)$.

$$\mathbf{round}(x)=x\cdot\gamma\tag{11}$$

Similarly, we convert a negative integer x to an inverse in the finite field $\mathbb{Z}_q$. This little trick has been widely used in previous works [56], [57], and we omit the converting process to default the parameters as integers in the following description for convenience.
5) *Byzantine Settings:* We compare the Byzantine-resilience aggregation rules in PriDFL with some traditional methods, e.g., Krum [17] and Trimmed mean [18]. Additionally, we refer to the setup of poisoning attacks in previous work, as follows:
Krum: Remarkably, we select a variant of Krum, m-Krum, as part of the comparison and made some minor modifications for the DFL scenario. Its main setup is that each

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.



(a) Wall clock times for SecAgg and PriDFL at dropout rate ρ=0%.

(b) Wall clock times for SecAgg and PriDFL at dropout rate ρ=10%.

(c) Wall clock times for SecAgg and PriDFL at dropout rate ρ=20%.

(d) Wall clock times for SecAgg and PriDFL at dropout rate ρ=30%.

Fig. 5. Comparison of running time for different schemes. Number of clients fixed $|\mathcal{U}| = 500$.

client calculates the sum of the Euclidean distances of the $|\mathcal{U}| - c - 2$ local models with neighboring nodes as a score, sorts the scores and then selects the lowest two ($m = 2$) local models for averaging as the global model.

Trimmed mean: Similarly, each client sorts the j-th parameter in all local models, trims the maximum and minimum values and then finds the mean as the j-th parameter of the global model.

Label flipping attack: We follow the setup of previous work by replacing the "1" class labels of the dataset with labels "7", i.e., the label flipping attack [56]. In this paper, we perform such attacks on both MNIST and CIFAR-10.

Sign flipping attack [58]: Replace the sign of selected gradient coordinates so that the aggregated update is driven in the wrong direction. In practice this can be implemented as a global sign flip on the entire local update or as a sparse sign flip that inverts only the largest-magnitude coordinates. A camouflage variant chooses which coordinates to flip so that the global sign histogram remains similar to a reference distribution while still corrupting key directions.

Model replacement attack [59]: Replace the submitted update with a scaled vector that steers the global model toward a target model or toward an attacker-chosen direction, optionally clipping the poisoned update norm to remain covert. The attacker computes a delta between the target (or local) model and the current global model, multiplies that delta by a scaling factor, optionally enforces a maximum norm relative to a benign update, and then submits the resulting poisoned update.

B. Comparison of Sharing Performance With Different Baselines

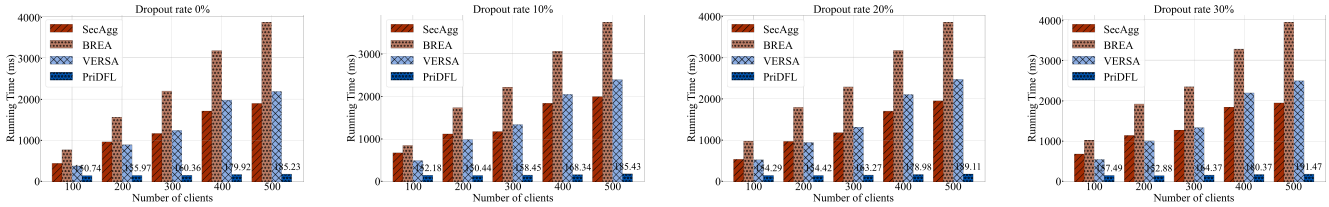
Recently, while plenty of literature focused on the SA protocol mentioned by SecAgg [8], [9], [10], [22], [25], [26], [27], [28], [29], it is noteworthy that none of these studies have changed the basic secret splitting approach (i.e., Shamir secret sharing). We focus on data masking and computing shares when comparing computational efficiency, which are the main differences between PriDFL and other Shamir secret sharing-based schemes. We follow the implementation details for *Faster-sharing* and the Shamir baseline outlined in section V-A. To avoid redundancy, we will not elaborate further here. We evaluate the computational cost of four schemes, SecAgg [5], BREA [46], VERSA [28] and PriDFL, for different data sizes, and as shown in Fig. 5. VERSA extends the verifiability of the aggregation protocol without changing the SecAgg data-sharing kernel, maintaining a comparable runtime to SecAgg. BERA, on the other hand, employs a sharing model that combines Feldman's verifiable secret sharing and Reed Solomon codes, which increases the extra computational cost to the client. Meanwhile, PriDFL based on *Faster-sharing* employs multi-secret sharing and NTT technology to reduce 6× less computation cost compared to others, which is especially superior when dealing with large-scale models. In addition, Fig. 6 shows the running time of the other three schemes with PriDFL for different numbers of clients with data fixed at 100 K entries. The runtime of our scheme is essentially stable at around 150 ms for different number of clients, which is better performance than the slowly increasing other schemes, especially with $10\times$ efficiency optimization for clients ≥ 500. We attribute the advantages of PriDFL to *Faster-sharing*, which combines Multi-Secret Sharing and NTT techniques. We assume that the secret vector size is fixed to m. Shamir secret sharing requires repeated split shares of a single secret, i.e., it needs to perform $m * n$ point-value evaluations. In contrast, *Faster-sharing* requires only $(m/t) * n$ interpolation operations, where t is the number of secrets shared each time, and n is a polynomial coefficient corresponding to the total number of clients.

C. Computational/communication Overhead of Each Module in PriDFL.

We followed SecAgg's evaluation setup and assumed the client sends a data vector with 3 bytes per entry. Meanwhile, we simulated on a Windows platform with Intel(R) Xeon(R) W-1390 @2.80 GHz 2.81 GHz, 64 GB of RAM. Fig. 7(a) and (b) show the wall-clock times of our scheme for handling various clients and data vector sizes at different dropout rates. It also shows that PriDFL does not suffer from dropouts when dealing with large-scale client participation and foundation models, and more suitable for decentralized multi-device participation scenarios. In Fig. 8, the communication increases slightly as the size of the data vector increases, which is tolerable.

In Tables IV and V, we give the computational and communication overheads of each module of PriDFL. Our protocol does not burden the clients with redundancy. For example, when the number of gradient vectors is set to 100 K, the total amount of communication transferred between clients is 3712.38 KB. Compared with the practical applications of FL, the deep neural network models are normally tens of MB or more. Moreover, our protocol does not suffer from client

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.



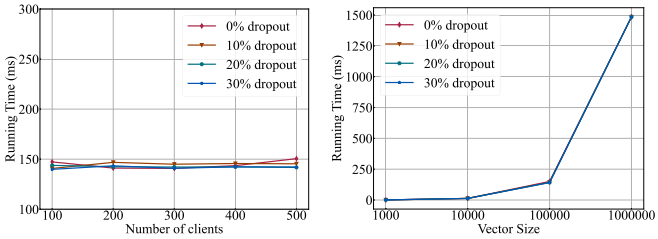
(a) Wall clock times for SecAgg and PriDFL at dropout rate ρ=0%

(b) Wall clock times for SecAgg and PriDFL at dropout rate ρ=10%

(c) Wall clock times for SecAgg and PriDFL at dropout rate ρ=20%

(d) Wall clock times for SecAgg and PriDFL at dropout rate ρ=30%

Fig. 6. Comparison of running time for different schemes. The data vector size is fixed to 100 K entries.



(a) Client runtime for drop rates on number of clients.

(b) Client runtime for drop rates on data vector size.

Fig. 7. Client running time for different dropout rates in number of clients and vector size.

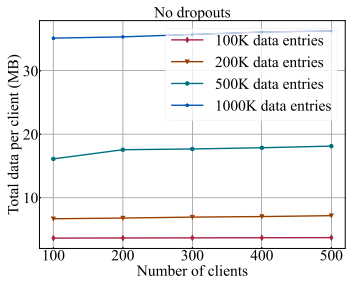


Fig. 8. Data transfer costs with no dropouts.

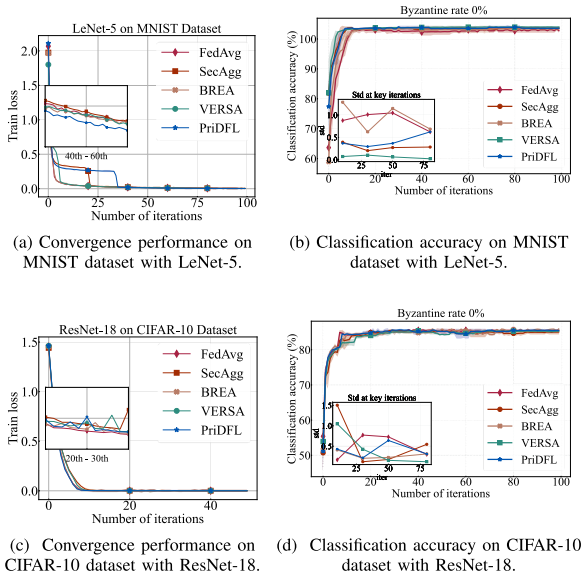
TABLE IV
COMPARISON OF FUNCTIONALITY, NOTE THAT WE OMIT THE TRANSMISSION DELAY BETWEEN CLIENTS. AND FIXED THE SIZE OF THE GRADIENT VECTOR TO 100 K AND THE NUMBER OF CLIENTS TO 500

	Dropout	AdvertiseKeys	Sharing	Masking	Unmasking	Total
Client	0%	1 ms	7373 ms	8043 ms	802 ms	16219 ms
Client	10%	1 ms	7449 ms	8079 ms	823 ms	16352 ms
Client	20%	1 ms	7671 ms	8083 ms	798 ms	16553 ms
Client	30%	1 ms	7662 ms	8395 ms	816 ms	16874 ms

TABLE V
COMMUNICATION OVERHEAD PER ROUND, THE GRADIENT VECTOR SIZE IS FIXED TO 100 K ENTRIES OF 24 BITS EACH, AND THE NUMBER OF CLIENTS IS SET TO 500

	Dropout	AdvertiseKeys	Sharing	Masking	Unmasking	Total
Client	0%	1.45 KB	3515.62 KB	3808.59 KB	196.78 KB	7522.44 KB
Client	10%	1.45 KB	3515.62 KB	3808.59 KB	196.78 KB	7522.44 KB
Client	20%	1.45 KB	3515.62 KB	3808.59 KB	196.78 KB	7522.44 KB
Client	30%	1.45 KB	3515.62 KB	3808.59 KB	196.78 KB	7522.44 KB

dropouts that affect the performance and maintains a similar runtime and communication as plaintext FL training.



(a) Convergence performance on MNIST dataset with LeNet-5.

(b) Classification accuracy on MNIST dataset with LeNet-5.

(c) Convergence performance on CIFAR-10 dataset with ResNet-18.

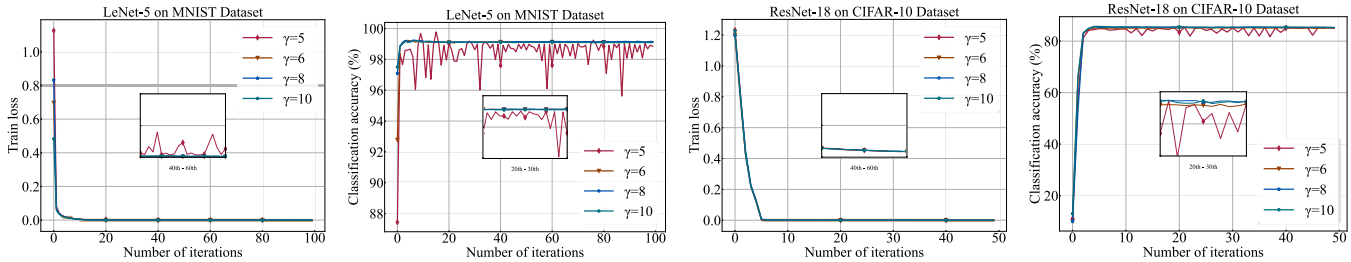
(d) Classification accuracy on CIFAR-10 dataset with ResNet-18.

Fig. 9. Classification accuracy on different datasets. Client dropout ratio fixed ρ=30% and Byzantine rate φ=0% .

D. Model Performance of Various Schemes Under Client Dropout Scenarios.

We adopt the FedAvg [51] aggregation algorithm without privacy protection as the baseline and compare SecAgg [5], BREA [46], VERSA [28], and PriDFL under identical experimental settings. In these schemes, inter-client interactions are represented by masked values, and only the aggregated global model is revealed after aggregation. Fig. 9 demonstrates that PriDFL achieves classification accuracy comparable to plaintext FL across different datasets and models, while simultaneously preserving local data privacy. The client dropout rate is fixed at 30% and the Byzantine rate at 0% for fair comparison. The results indicate that the masking mechanism does not significantly affect either prediction accuracy or convergence behavior, providing evidence that lightweight masking can be effectively generalized to real-world federated model applications. The shaded regions in Fig. 9 represent the standard deviation across three independent runs, reflecting the training stability of each method. PriDFL consistently exhibits the smallest variance throughout training, which confirms its robustness and stability even under partial participation. The embedded subplots further illustrate the standard deviation at key iterations, where PriDFL

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.



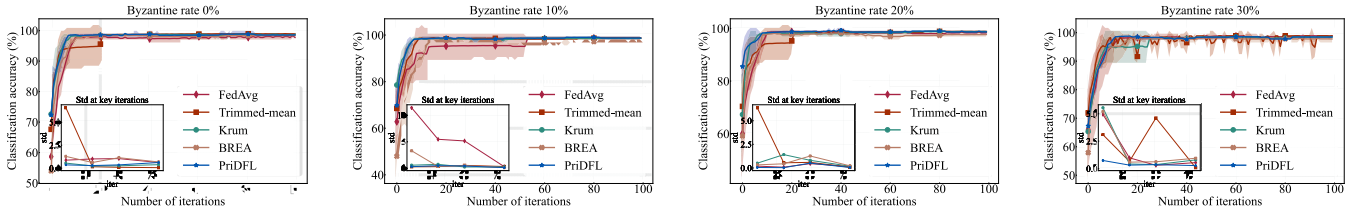
(a) Convergence efficiency with LeNet-5 on MNIST dataset scaling factor is set at $\gamma = 5, 6, 8, 10$ respectively.

(b) Classification accuracy with LeNet-5 on MNIST dataset scaling factor is set at $\gamma = 5, 6, 8, 10$ respectively.

(c) Convergence efficiency with ResNet-18 on CIFAR-10 dataset scaling factor is set at $\gamma = 5, 6, 8, 10$ respectively.

(d) Classification accuracy with ResNet-18 on CIFAR-10 dataset scaling factor is set at $\gamma = 5, 6, 8, 10$ respectively.

Fig. 10. Comparison of different data truncations on classification accuracy in PriDFL. Client dropout ratio fixed ρ=30% .



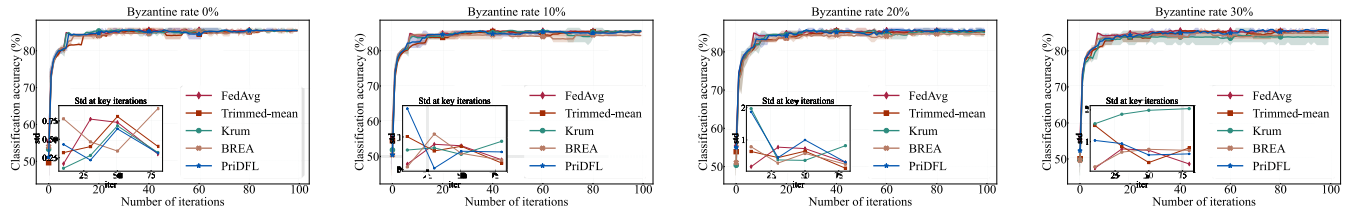
(a) Performance of aggregation resilience at Byzantine rate φ=0%.

(b) Performance of aggregation resilience at Byzantine rate φ=10%.

(c) Performance of aggregation resilience at Byzantine rate φ=20%.

(d) Performance of aggregation resilience at Byzantine rate φ=30%.

Fig. 11. Classification accuracy of different schemes on MNIST dataset with LeNet5. Number of clients fixed $|\mathcal{U}| = 100$ and drop rate fixed ρ=30% . The inset illustrates the standard deviation for key rounds across different schemes.



(a) Performance of aggregation resilience at Byzantine rate φ= 0%.

(b) Performance of aggregation resilience at Byzantine rate φ= 10%.

(c) Performance of aggregation resilience at Byzantine rate φ= 20%.

(d) Performance of aggregation resilience at Byzantine rate φ= 30%.

Fig. 12. Classification accuracy of different schemes on CIFAR-10 dataset with ResNet-18. Number of clients fixed $|\mathcal{U}| = 100$ and dropout rate fixed ρ=30% . The inset illustrates the standard deviation for key rounds across different schemes.

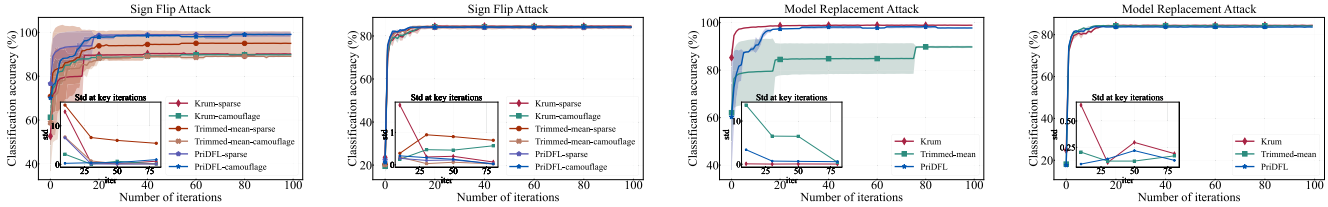
maintains a consistently low std compared with SecAgg, BREA, and VERSA, particularly in later rounds of training. This trend indicates that PriDFL's aggregation process converges smoothly with reduced fluctuation in model accuracy. The convergence curves of train loss on both datasets also show that PriDFL reaches the same or better minima as plaintext FedAvg, suggesting that the masking and secure aggregation operations introduce negligible overhead in optimization.

Specifically, we further examine the influence of data truncation on PriDFL performance. As shown in Fig. 10, the classification accuracy of PrDFL shows fluctuation when the truncation factor $\gamma = 5$, while the accuracy curve tends to be stable when $\gamma \geq 8$. Therefore, in this paper, we fix $\gamma = 8$ to ensure the high performance of the model at PriDFL training. In addition, the threshold property of *Faster-sharing* also enables our PriDFL to support client dropouts during aggregation.

E. Classification Accuracy of PriDFL Versus Baselines Under Different Poisoning Attacks.

We evaluated the Byzantine-resilient aggregation rules in PriDFL with two convolutional neural network models on different datasets, using the unpoisoned FedAvg as a baseline. Figs. 11 and 12 illustrate the impact of varying Byzantine client ratios on model accuracy. In Fig. 11(c) and (d), corresponding to Byzantine rates of 20% and 30% , PriDFL consistently maintains approximately 95% accuracy, while the fluctuations of the accuracy curve are attributed to the sensitivity of KL-divergence to extreme values and the inherent randomness of neural network updates. The shaded regions represent the standard deviation across three independent runs, indicating the degree of variance during training. Compared with other schemes, PriDFL exhibits the narrowest shaded areas, reflecting lower variance and more

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.



(a) Classification accuracy with LeNet-5 on the MNIST dataset under sign flipping attacks.

(b) Classification accuracy with ResNet-18 on the CIFAR-10 dataset under sign flipping attacks.

(c) Classification accuracy with LeNet-5 on the MNIST dataset under model replacement attacks.

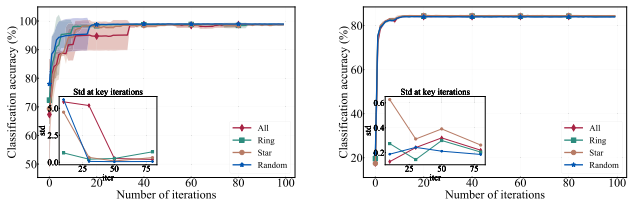
(d) Classification accuracy with ResNet-18 on the CIFAR-10 dataset under model replacement attacks.

Fig. 13. Comparison of classification accuracy between PriDFL and baseline under different poisoning attacks. Client Byzantine rate fixed φ=30% .

stable convergence under Byzantine perturbations. The embedded subplots further display the variation of standard deviation at key iterations, showing that the std of PriDFL decreases steadily with the progress of training, while the baselines such as FedAvg and Krum maintain higher and more irregular variance, especially at higher Byzantine ratios.

At the same Byzantine ratio, the model accuracies in Figs. 12(c) and (d) on the CIFAR-10 dataset remain close to those of the unpoisoned FedAvg, confirming the robustness of PriDFL. Notably, compared with BREA [46], which enhances Byzantine resilience through domain mapping in secure aggregation, PriDFL achieves 2-3% higher inference accuracy and faster model convergence. BREA maps gradient shares into a designated domain space and measures Euclidean distances within that space; however, such a domain-mapping approach compromises the precision of element-wise distance computation and neglects the sign characteristics of the original gradients. In contrast, PriDFL's Byzantine aggregation rule effectively preserves directional information and suppresses adversarial deviations, resulting in improved robustness and stability. Furthermore, the method generalizes seamlessly to non-IID data settings with consistently high performance and minimal variance across multiple runs.

Fig. 13 illustrates the classification accuracy of PriDFL and baseline aggregation schemes under two representative poisoning attacks, including sign-flipping and model-replacement attacks, across MNIST and CIFAR-10 datasets. The shaded areas around each curve represent the standard deviation of three independent runs, which reflect the stability of each method during training. A narrower shaded region indicates smaller variance and higher robustness to random fluctuations. On the MNIST dataset with LeNet-5, PriDFL exhibits a stable and rapid convergence trend under both sign-flipping and model-replacement attacks. The shaded regions in Fig. 13 represent the standard deviation over three independent runs, reflecting the stability of each aggregation method during training. The fluctuation range of PriDFL remains smaller than those of Krum and Trimmed-mean throughout the iterations, indicating that PriDFL effectively mitigates the instability introduced by malicious gradient perturbations. In particular, under sign-flipping attacks, PriDFL rapidly achieves a high and steady accuracy while maintaining a low variance, whereas the baselines display noticeable oscillations in early training. This stability is reinforced by the evolution of standard deviation in the embedded subplots, where PriDFL shows a continuous decline in variance, implying that its aggregation strategy progressively stabilizes model updates. The advantage of PriDFL under sign-flipping attacks arises from its incorporation of the SignGuard principle, which filters and neutralizes adversarial sign manipulations in client gradients, leading to improved robustness. A consistent pattern is also observed on the CIFAR-10 dataset with ResNet-18, where PriDFL maintains accuracy comparable to the clean baseline even with 30% Byzantine clients, and its shaded area remains markedly narrower than those of Krum and Trimmed-mean. The smaller standard deviations across key iterations demonstrate that PriDFL's adaptive and privacy-preserving aggregation design sustains consistent learning behavior under adversarial perturbations. Overall, PriDFL achieves the highest accuracy with minimal variance across both datasets and attack types, confirming its superior resilience and convergence stability in Byzantine and poisoning environments.



(a) Classification accuracy with LeNet-5 on the MNIST dataset under different topologies.

(b) Classification accuracy with ResNet-18 on the CIFAR-10 dataset under different topologies.

Fig. 14. Comparison of PriDFL classification accuracy under different topologies. Client dropout rate and Byzantine rate are fixed ρ=30% and φ=10% , respectively.

F. Model Performance of PriDFL Under Different Topologies.

Fig. 14 presents the classification accuracy of PriDFL under different communication topologies on MNIST and CIFAR-10 datasets. The shaded regions represent the standard deviation over three independent runs, reflecting the stability of the model under each topology. PriDFL maintains stable convergence across all structures, demonstrating strong adaptability to varying network connectivity patterns. On the MNIST dataset with LeNet-5, the fully connected topology (labeled "all" in the diagram) achieves the highest accuracy and the smallest variance due to full connectivity that facilitates efficient model information exchange. The ring and star topologies converge slightly slower but eventually approach the fully connected

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
COMMUNICATION OVERHEAD FOR DIFFERENT COMMUNICATION TOPOLOGIES, THE GRADIENT VECTOR SIZE IS FIXED TO 100 K ENTRIES OF 24 BITS EACH, AND THE NUMBER OF CLIENTS IS SET TO 500

	Topology	AdvertiseKeys	Sharing	Masking	Unmasking	Total
Client	All	1.45 KB	236.10 MB	243.05 MB	95.36 MB	574.52 MB
Client	Ring	1.45 KB	1271.87 KB	1285.93 KB	190.62 KB	2749.87 KB
Client	Star	1.45 KB	3859.37 KB	3929.68 KB	195.31 KB	7985.81 KB
Client	Random	1.45 KB	3515.62 KB	3808.59 KB	196.78 KB	7522.44 KB

case, while the random topology achieves comparable final accuracy with moderate variance caused by uneven neighbor interactions. The embedded subplots further show that the standard deviation of PriDFL steadily decreases with training progress under all topologies, indicating increasing stability of the aggregation process. Similar observations are made on the CIFAR-10 dataset with ResNet-18, confirming that PriDFL consistently achieves robust and stable performance regardless of the network structure.

Table VI compares the communication overhead of PriDFL under different network topologies, where the gradient vector size is fixed to 100 K entries of 24 bits and the number of clients is 500. The all topology exhibits the highest total communication overheads of approximately 574.52 MB per client, since each client must exchange masked gradients with all remaining $n-1$ peers in every round. This full connectivity ensures maximum aggregation accuracy but incurs substantial bandwidth consumption. In contrast, the ring topology significantly reduces the overhead to 2749.87 KB, as each client only communicates with its two direct neighbors. The star and random topologies, where each client maintains 10 neighbors, incur intermediate communication overheads of 7985.81 KB and 7522.44 KB, respectively. Although their per-round bandwidth is higher than in the ring structure, they provide faster model synchronization and more stable aggregation due to richer neighborhood information. Across all configurations, the **Sharing** and **Masking** phases dominate total transmission volume, while **AdvertiseKeys** and **Unmasking** remain negligible. Overall, the results indicate that PriDFL scales efficiently with varying degrees of connectivity. While fully connected networks deliver the strongest consensus, sparse or partially connected topologies achieve a favorable balance between communication efficiency and convergence stability, confirming the practicality of PriDFL in resource-constrained decentralized settings.

VIII. DISCUSSION AND LIMITATIONS

A. Limitations of Trusted Authorities

In practical consortium scenarios such as industrial alliances, healthcare collaborations, or V2X ecosystems, introducing a lightweight, consortium-operated KGC aligns naturally with existing PKI and identity governance mechanisms. In PriDFL, the KGC serves only as a one-time trusted setup entity that binds identities, certifies public parameters, and optionally publishes a random initialization for the global model. This design preserves the fully decentralized training topology and minimizes the online attack surface. Nevertheless, the existence of a centralized KGC introduces potential vulnerabilities. If compromised or malicious, the KGC could leak or bias cryptographic material, Sybil attacks, issue inconsistent parameters that cause desynchronization, or publish a tainted initialization that embeds hidden triggers. These issues, although limited to the setup phase, undermine confidentiality, authenticity, and availability guarantees, highlighting a residual trust assumption in PriDFL's initialization stage.

To alleviate these risks, PriDFL can incorporate several complementary safeguards. A distributed or threshold-based setup can substitute the single KGC with a rotating t-of-n committee so that no single entity possesses unilateral control. Public verifiability can be enhanced by recording the issued parameters and revocation records on an append-only transparency log, such as a blockchain-based registry, which allows all clients to audit parameter generation. Ephemeral and forward-secure masking keys combined with periodic re-keying further improve key hygiene, while binding credentials to attested devices and introducing rate-limiting mechanisms help prevent large-scale Sybil registration. In addition, clients can conduct initialization sanity checks by verifying the statistical distribution of the published weights or by performing short validation runs on local held-out data to detect anomalous or poisoned model initializations. Through these measures, PriDFL substantially reduces the trust burden on the KGC and renders the setup phase more transparent, verifiable, and resilient in real-world deployments.

B. Limitations of the Training Framework

Although the sign-distribution validator guided by the KL divergence offers a lightweight and privacy-preserving criterion, the raw KL value may fluctuate when some sign categories rarely occur. PriDFL mitigates this issue by adopting additive smoothing on the three-category sign histogram, by computing the KL divergence against an exponential moving average of recent sign distributions, and by clipping the per-client KL value followed by weight renormalization under the masked-sum protocol. These steps reduce variance without disclosing any unmasked statistics, since only blinded KL values are aggregated and only the final sum is unblinded for weight normalization.

PriDFL's defense can be challenged by adaptive adversaries that craft gradients whose sign distribution closely matches the benign reference while embedding harmful magnitudes or sparse targeted flips. In such cases the KL signal may remain small and the weight may not drop significantly. To reduce this residual risk, PriDFL can combine the sign-based validator with complementary checks that remain compatible with our privacy and decentralization constraints, including per-update norm bounding before masking, per-coordinate sign-flip rate control across rounds via temporal consistency tests, and reputation-based down-weighting that decays the influence of clients whose past updates repeatedly trigger post-aggregation inconsistency. These measures preserve the masking workflow and operate locally, while the masked KL summation and single-mask aggregation remain unchanged.

Beyond the trust assumption in the setup phase, PriDFL inherits two training-time limitations. The first limitation is that the validator's effectiveness depends on reliable local estimation

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

of sign distributions when data are extremely scarce or highly skewed. The proposed smoothing, temporal averaging, and clipping alleviate this issue by stabilizing probabilities and by bounding per-round influence. The second limitation is that repeated local validation introduces additional computation on resource-constrained devices. A practical improvement is to subsample coordinates for sign statistics and to compute the exponential moving average at a lower cadence, which reduces cost while retaining robustness. These points complement the earlier discussion on training framework limitations and provide concrete mitigation paths within PriDFL's current design.

IX. Conclusion

In this paper, we introduced a novel paradigm for secure and efficient decentralized federated learning, PriDFL, which protects model privacy while supporting Byzantine- resilience. We presented Faster-sharing, a faster secret-sharing scheme to reduce the computational cost on the client. We proposed a Byzantine resilient aggregation criterion based on model cross-updating to guarantee correct model convergence. To protect gradient privacy in DFL, we developed a single mask structure in random communication. Security and experimental analysis demonstrate the practicality of our approach. As part of our research work, we will consider consistency checking of the global model for malicious user participation scenarios and incorporate verifiable computations to achieve verifiability of aggregation results.

References

[1] S. Savazzi, M. Nicoli, M. Bennis, S. Kianoush, and L. Barbieri, "Opportunities of federated learning in connected, cooperative, and automated industrial systems," *IEEE Commun. Mag.*, vol. 59, no. 2, pp. 16–21, Feb. 2021.

[2] T. Wang, Y. Liu, X. Zheng, W. Jia, and M. Xie, "Edge-based communication optimization for distributed federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2015–2024, Jul./Aug. 2021.

[3] E. T. M. Beltrán et al., "Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges," *IEEE Commun. Surv. Tut.*, vol. 25, no. 4, pp. 2983–3013, Fourthquarter 2023.

[4] X. Cao, M. Fang, J. Liu and et al., "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021.

[5] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.

[6] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[7] S. Kadhe et al., "FastSecAgg: Scalable secure aggregation for privacy-preserving federated learning," 2020, *arXiv:2009.11248*.

[8] J. H. Bell et al., "Secure single-server aggregation with (poly) logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1253–1269.

[9] X. Guo et al., "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 1736–1751, 2020.

[10] Y. Wang, A. Zhang, S. Wu, and S. Yu, "VOSA: Verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3601–3616, Sep./Oct. 2023.

[11] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "ELSA: Secure aggregation for federated learning with malicious actors," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 1961–1979.

[12] H. Lycklama, L. Burkhalter, A. Viand, N. Küchler, and A. Hithnawi, "RoFL: Robustness of secure federated learning," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 453–476.

[13] N. Peri et al., "Deep K-Nn Defense Against Clean-Label Data Poisoning Attacks," in *Proc. Workshops Comput. Vis.*, 2020, pp. 55–70.

[14] R. Schuster et al., "You autocomplete me: Poisoning vulnerabilities in neural code completion," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1559–1575.

[15] L. Zhao et al., "Shielding collaborative learning: Mitigating poisoning attacks through client-side detection," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2029–2041, Sep./Oct. 2021.

[16] M. Ren et al., "Learning to reweight examples for robust deep learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4334–4343.

[17] P. Blanchard et al., "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inform. Process. Syst.*, 2017, p. 30.

[18] D. Yin et al., "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.

[19] Y.-C. Lai et al., "Two-phase defense against poisoning attacks on federated learning-based intrusion detection," *Comput. Secur.*, vol. 129, 2023, Art. no. 103205.

[20] A. Roy Chowdhury et al., "Eiffel: Ensuring integrity for federated learning," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2535–2549.

[21] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2429–2443.

[22] L. Zhao, J. Jiang, B. Feng, Q. Wang, C. Shen, and Q. Li, "SEAR: Secure and efficient aggregation for Byzantine-robust federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3329–3342, Sep./Oct. 2022.

[23] A. Moustafa et al., "PPCSA: Partial participation-based compressed and secure aggregation in federated learning," in *Proc. Adv. Inf. Netw. Appl.: Proc. 35th Int. Conf. Adv. Inf. Netw. Appl.*, 2021, pp. 345–357.

[24] M. Mohamad et al., "Sok: Secure aggregation based on cryptographic schemes for federated learning," in *Proc. Privacy Enhancing Technol. Symp.*, 2023, pp. 140–157.

[25] J. So, B. Güler, and A. S. Avestimehr, "Turbo-Aggregate: Breaking the quadratic aggregation barrier in secure federated learning," *IEEE J. Sel. Areas Inf. Theory*, vol. 2, no. 1, pp. 479–489, Mar. 2021.

[26] Z. Liu, J. Guo, K.-Y. Lam, and J. Zhao, "Efficient dropout-resilient aggregation for privacy-preserving machine learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1839–1854, 2023.

[27] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "Verifynet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2019.

[28] C. Hahn, H. Kim, M. Kim, and J. Hur, "VerSA: Verifiable secure aggregation for cross-device federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 36–52, Jan./Feb. 2023.

[29] T. Eltaras, F. Sabry, W. Labda, K. Alzoubi, and Q. Ahmedeltaras, "Efficient verifiable protocol for privacy-preserving aggregation in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2977–2990, 2023.

[30] M. Joye and B. Libert, "A scalable scheme for privacy-preserving aggregation of time-series data," in *Proc. 17th Int. Conf. Financial Cryptography Data Secur.*, Okinawa, Japan, 2013, pp. 111–125.

[31] F. Benhamouda et al., "A new framework for privacy-preserving aggregation of time-series data," in *Proc. ACM Trans. Inf. System Secur.*, 2016, pp. 1–21.

[32] M. Abdalla et al., "Multi-input inner-product functional encryption from pairings," in *Proc. Adv. Cryptology–EUROCRYPT 36th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Paris, France, 2017, pp. 601–626.

[33] R. Xu et al., "Hybridalpha: An efficient approach for privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur.*, 2019, pp. 13–23.

[34] D. Wu, M. Pan, Z. Xu, Y. Zhang, and Z. Han, "Towards efficient secure aggregation for model update in federated learning," in *Proc. IEEE Glob. Commun. Conf.*, 2020, pp. 1–6.

[35] T. Nguyen et al., "A novel decentralized federated learning approach to train on globally distributed, poor quality, and protected private medical data," *Sci. Rep.*, vol. 12, no. 1, 2022, Art. no. 8888.

[36] S. Chen, D. Yu, Y. Zou, J. Yu, and X. Cheng, "Decentralized wireless federated learning with differential privacy," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6273–6282, Sep. 2022.

[37] I. Arapakis et al., "P4l: Privacy preserving peer-to-peer learning for infrastructureless setups," 2023, *arXiv:2302.13438*.

[38] D. Pasquini, M. Raynal, and C. Troncoso, "On the (in) security of peer-to-peer decentralized machine learning," in *Proc. IEEE Symp. Secur. Privacy,*2023, pp. 418–436.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.

[39] M. Fang et al., "Local model poisoning attacks to {Byzantine-Robust} federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.
[40] S. Shen, S. Tople, and P. Saxena, "Auror: Defending against poisoning attacks in collaborative deep learning systems," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, 2016, pp. 508–519.
[41] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *Proc. 23rd Int. Symp. Res. Attacks Intrusions Defenses*, 2020, pp. 301–316.
[42] M. Franklin and M. Yung, "Communication complexity of secure computation," in *Proc. 24th Annu. ACM Symp. Theory Comput.*, 1992, pp. 699–710.
[43] P. Longa and M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," in *Proc. Cryptology Netw. Secur.: 15th Int. Conf.*, 2016, pp. 124–139.
[44] W. Diffie and M. E. Hellman, "New directions in cryptography," in *Proc. Democratizing Cryptography: Work Whitfield Diffie Martin Hellman*, 2022, pp. 365–390.
[45] T. Jahani-Nezhad, M. A. Maddah-Ali, S. Li, and G. Caire, "Swiftagg+ : Achieving asymptotically optimal communication loads in secure aggregation for federated learning," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 977–989, Apr. 2023.
[46] J. So, B. Güler, and A. S. Avestimehr, "Byzantine-resilient secure federated learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2168–2181, Jul. 2021.
[47] M. Dahl, V. Pastro, and M. Poumeyrol, "Private data aggregation on a budget," *Cryptology ePrint Arch.*, vol. 643, 2017, Art. no. 2632.
[48] X. Lian et al., "Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent," in *Proc. Adv. Neural Inform. Process. Syst.*, 2017, pp. 5331–5341.
[49] J. Xu, S.-L. Huang, L. Song, and T. Lan, "Byzantine-robust federated learning through collaborative malicious gradient filtering," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst.*, 2022, pp. 1223–1235.
[50] G. Baruch, M. Baruch, and Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Proc. Adv. Neural Inform. Process. Syst.*, 2019, pp. 8632–8642.
[51] B. McMahan, E. Moore, D. Ramage, B. C. Ooi, Y. Wu, and H. Yu, "Communication-efficient learning of deep networks from decentralized data," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2017, pp. 1273–1282.
[52] Y. LeCun, C. Cortes, and C. J. Burges, "Mnist handwritten digit database," 1998. [Online]. Available: http://yann.lecun.com/exdb/mnist/
[53] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Tech. Rep., Univ. of Toronto, 2009.
[54] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[55] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. pattern Recognit.*, 2016, pp. 770–778.
[56] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1639–1654, 2022.
[57] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, and R. H. Deng, "Privacy-preserving federated deep learning with irregular users," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1364–1381, Mar./Apr. 2020.
[58] F. Colosimo and F. De Rango, "Dynamic gradient filtering in federated learning with byzantine failure robustness," *Future Gener. Comput. Syst.*, vol. 160, pp. 784–797, 2024.
[59] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2938–2948.



Shuai Wang received the BSc degree in software engineering from Fujian Normal University, Fuzhou, China, in 2020. He is currently working toward the PhD degree with the College of Computer Science and Technology, Guizhou University. His research interests include deep federated learning and privacy protection.



Youliang Tian (Senior Member, IEEE) received the PhD degree in cryptography from Xidian University, Xi'an, China, in 2012. He is currently a full professor and PhD supervisor with the State Key Laboratory of Public Big Data and the College of Computer Science and Technology, Guizhou University. He has authored more than 100 publications and 2 books. His current research interests include algorithmic game theory, cryptography and security protocols, privacy protection, and blockchain.



Jinbo Xiong (Senior Member, IEEE) received the PhD degree in computer system architecture from Xidian University, China, in 2013. He is currently the full professor and Ph.D. supervisor with the Fujian Provincial Key Laboratory of Network Security and Cryptology, and College of Computer and Cyber Security, Fujian Normal University. He has authored more than 100 publications, 3 books and authorized 7 invention patents. His research interests include secure deep learning, data security, privacy protection, and Internet of Things.



Renwan Bi (Member, IEEE) received the PhD degree in cyberspace security with the College of Computer and Cyber Security, Fujian Normal University. He is currently an assistant professor with the School of Cyber Science and Engineering, Minjiang University. He has authored or coauthored more than 20 papers in journals and conferences, including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Internet of Things Journal*, *IEEE Network*, IEEE International Conference on Communications, and so on. His research interests include data privacy protection and artificial intelligence security.



Jianfeng Ma (Member, IEEE) received the BS degree in computer science from Shaanxi Normal University, in 1982, and MS degree in computer science from Xidian University, in 1992, and the PhD degree in computer science from Xidian University, in 1995. He is currently a professor with the School of Cyber Engineering, Xidian University. He has authored or coauthored more than 150 journal and conference papers. His research interests include information security, cryptography, and network security.



Yan Zhang (Fellow, IEEE) received the PhD degree from the School of Electrical and Electronics Engineering, Nanyang Technological University, Singapore. He is currently a full professor with the Department of Informatics, University of Oslo, Oslo, Norway. His research interests include nextgeneration wireless networks leading to 5G and cyber-physical systems. He is a fellow of IET and an elected member of Academia Europaea, the Royal Norwegian Society of Sciences and Letters (DKNVS), and Norwegian Academy of Technological Sciences. He is an editor for several IEEE publications, including *IEEE Communications Magazine*, *IEEE Network*, *IEEE Transactions on Green Communications and Networking*, *IEEE Communications Surveys and Tutorials*, and *IEEE Internet of Things Journal*.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:47 UTC from IEEE Xplore. Restrictions apply.